

MANUAL DE EXPERIMENTOS
LINGUAGEM PYTHON

PARTE IV

Ramon Mayor Martins, Ph.D.
ramon.mayor at ifsc.edu.br
IFSC · Câmpus São José

@rmayormartins · [rmayormartins.github.io](https://github.com/rmayormartins)

IA generativa e modelos de fundação
Versão 2026/2

XVIII. FUNDAMENTOS DA GERAÇÃO

45	Panorama da IA generativa	2
As famílias de modelos generativos · O que este volume cobre		
46	Tokenização e embeddings	6
Tokenização · Embeddings		

XIX. A ARQUITETURA TRANSFORMER

47	Atenção e transformers	6
A conta por dentro · Transformer no Keras		

XX. FAMÍLIAS GENERATIVAS

48	Autoencoder variacional e GAN	4
Autoencoder variacional · Redes adversariais		
49	Modelos de difusão	3
50	Geração autorregressiva	4

XXI. SISTEMAS COM MODELOS DE LINGUAGEM

51	Modelos de linguagem e prompting	5
Chamando um modelo por código · Prompting		
52	RAG: geração aumentada por recuperação	6
Indexação · Consulta		
53	Ajuste fino, LoRA e quantização	5
LoRA · Quantização		

XXII. APÊNDICES

G	Cartão de consulta e glossário	1
Glossário · Qual família para qual tarefa · Antes de publicar algo generativo · Para continuar		

[0] SOBRE ESTA PARTE IV

O que veio depois da convolução: atenção, transformers e as arquiteturas que geram texto, imagem e som.

A Parte III termina na seção 44, com o modelo treinado, avaliado e publicado. Este volume continua na **seção 45** e trata de uma família diferente de problema: em vez de classificar ou prever um número, **gerar** algo novo que se pareça com os dados de treino.

O percurso

A seção 45 apresenta o panorama e o que muda quando a saída é uma amostra em vez de um rótulo. As seções 46 e 47 constroem a representação: tokenização, embeddings e o mecanismo de atenção que sustenta todos os modelos atuais. As seções 48 a 50 percorrem as famílias generativas, do autoencoder variacional e da GAN aos modelos de difusão e à geração autorregressiva. As seções 51 a 53 fecham com o uso prático: modelos de linguagem e prompting, RAG e ajuste fino com LoRA e quantização.

O que é preciso saber antes

Este volume assume as seções 38 a 41 da Parte III, isto é, redes neurais, Keras e o treino de arquiteturas profundas. Assume também NumPy e Matplotlib, da Parte II.

O que instalar

[>] TERMINAL

```
pip install tensorflow-cpu scikit-learn matplotlib
```

[!] MODELOS PEQUENOS, DE PROPÓSITO

Nenhum exemplo aqui baixa um modelo pré-treinado de bilhões de parâmetros: tudo é construído do zero, em escala reduzida, para rodar em CPU comum em segundos ou poucos minutos. Um transformer com dez mil parâmetros treinado em dez frases não escreve poesia, mas usa exatamente o mesmo mecanismo dos modelos grandes, e permite abrir cada etapa e olhar por dentro. Onde o uso real depende de uma API ou de um modelo grande, o código aparece sem painel de saída e assim sinalizado.

=====

|| BLOCO XVIII

FUNDAMENTOS DA GERAÇÃO

O que é diferente quando o modelo produz conteúdo em vez de escolher uma classe.

=====

```
=====
[ 45 ] PANORAMA DA IA GENERATIVA
=====
```

O que muda quando o modelo deixa de classificar e passa a produzir. O processo das seis etapas continua valendo, com três diferenças que mudam tudo.

Um modelo **discriminativo**, como os das Partes III, aprende a fronteira entre classes: dado **x**, qual é **y**. Um modelo **generativo** aprende como os dados são feitos: ele estima a distribuição e sabe produzir amostras novas que poderiam ter vindo dela.

+++ #CÓDIGO 45.1 As seis etapas na IA generativa -----

O processo é o mesmo; o que muda é o conteúdo de cada caixa. A etapa 4 é a que mais dói: não existe uma resposta certa contra a qual comparar.

```
ETAPAS = [
    ("1", "REQUISITOS",
     "não é acurácia: é utilidade, custo e risco"),
    ("2", "DADOS",
     "corpus, licença, tokenização e representação"),
    ("3", "TREINAMENTO",
     "pré-treino caríssimo, ajuste fino barato"),
    ("4", "AVALIAÇÃO",
     "sem gabarito único: perplexidade, humano, juiz"),
    ("5", "PREDIÇÃO",
     "geração com amostragem, e o contexto que você monta"),
    ("6", "EXPORTAÇÃO",
     "quantização, servidor, custo por token"),
]

MUDA = {
    "1": "o erro não é binário, é uma resposta ruim",
    "2": "o dado é o próprio texto, sem rótulo humano",
    "3": "quase ninguém treina do zero: adapta",
    "4": "duas respostas diferentes podem estar certas",
    "5": "a mesma entrada pode dar saídas diferentes",
    "6": "o modelo raramente cabe na sua máquina",
}

LARGURA = 56
for numero, titulo, detalhe in ETAPAS:
    print("+" + "=" * (LARGURA - 2) + "+")
    print("| " + f"{numero} {titulo}".ljust(LARGURA - 4) + " |")
    print("| " + f" {detalhe}".ljust(LARGURA - 4) + " |")
    print("| " + f" >> {MUDA[numero]}".ljust(LARGURA - 4) + " |")
    print("+" + "=" * (LARGURA - 2) + "+")
```

```
--- SAÍDA ---
+=====+
| 1 REQUISITOS |
| não é acurácia: é utilidade, custo e risco |
| >> o erro não é binário, é uma resposta ruim |
+=====+
| 2 DADOS |
| corpus, licença, tokenização e representação |
| >> o dado é o próprio texto, sem rótulo humano |
+=====+
| 3 TREINAMENTO |
| pré-treino caríssimo, ajuste fino barato |
| >> quase ninguém treina do zero: adapta |
+=====+
| 4 AVALIAÇÃO |
| sem gabarito único: perplexidade, humano, juiz |
| >> duas respostas diferentes podem estar certas |
+=====+
| 5 PREDIÇÃO |
| geração com amostragem, e o contexto que você monta |
| >> a mesma entrada pode dar saídas diferentes |
+=====+
| 6 EXPORTAÇÃO |
| quantização, servidor, custo por token |
| >> o modelo raramente cabe na sua máquina |
+=====+
```

+++ As famílias de modelos generativos

ASSUNTO	SEÇÃO
tokenização e embeddings	46
atenção e transformers	47
autoencoder variacional e GAN	48
difusão	49
geração autorregressiva de sequências	50
modelos de linguagem, prompting e agentes	51
RAG: geração aumentada por recuperação	52
ajuste fino, LoRA e quantização	53
avaliação de sistemas generativos	54
limites, riscos e uso responsável	55

+-- Tabela 45.2: o mapa deste volume.

[i] O QUE RODA AQUI E O QUE NÃO RODA

Todos os modelos deste volume são pequenos e foram treinados do zero na geração do arquivo, em CPU. Não há pesos baixados de nenhum repositório: o objetivo é entender o mecanismo, não competir com um modelo de fronteira. Onde o código depende de um serviço externo, como na seção 51, ele aparece sem painel de saída e está marcado como não executado.

-
- >> EXPERIMENTOS PROPOSTOS
- . Explique, em uma frase cada, a diferença entre um modelo que classifica e um que gera.
 - . Preencha as seis caixas do CÓDIGO 45.1 para um problema generativo do seu interesse.
 - . Gere cinco amostras sintéticas de classes diferentes com o naive bayes e avalie visualmente a qualidade.
 - . Para cada família da Tabela 45.1, aponte uma aplicação em telecomunicações.

[46] TOKENIZAÇÃO E EMBEDDINGS

Antes de qualquer geração, o texto precisa virar número. As duas decisões desta seção definem o que o modelo é capaz de aprender.

Tokenização

+-+ #CÓDIGO 46.1 Três granularidades -----

Tokenizar por palavra explode o vocabulário e quebra em qualquer termo novo. Por caractere, o modelo gasta capacidade aprendendo a soletrar.

```
frase = "o transformador aquece rapidamente"

por_caractere = list(frase)
por_palavra = frase.split()

print("caracteres:", len(por_caractere), "tokens |",
      len(set(por_caractere)), "símbolos distintos")
print("palavras: ", len(por_palavra), "tokens |",
      len(set(por_palavra)), "símbolos distintos")
print()
print("por caractere: vocabulário minúsculo, sequências longas")
print("por palavra:  sequências curtas, vocabulário imenso")
print("subpalavra:   o meio-termo que todos usam hoje")
```

--- SAÍDA ---

```
caracteres: 34 tokens | 16 símbolos distintos
palavras:   4 tokens | 4 símbolos distintos
```

```
por caractere: vocabulário minúsculo, sequências longas
por palavra:   sequências curtas, vocabulário imenso
subpalavra:    o meio-termo que todos usam hoje
```

+-----

--- #CÓDIGO 46.2 BPE: aprendendo o vocabulário a partir do texto -----

O algoritmo funde os pares mais frequentes, rodada após rodada. Radicais comuns viram um token só, e palavras raras continuam decompostas: é assim que um vocabulário fixo cobre qualquer texto.

```
from collections import Counter

CORPUS = ("antena antenas antenado transmissor transmissores "
         "receptor receptores transmitir receber ") * 30

def preparar(corpus):
    """Cada palavra vira uma lista de caracteres com marca de fim."""
    contagem = Counter(corpus.split())
    return {tuple(list(p) + ["_"]): n for p, n in contagem.items()}

def pares_frequentes(palavras):
    pares = Counter()
    for simbolos, n in palavras.items():
        for i in range(len(simbolos) - 1):
            pares[(simbolos[i], simbolos[i + 1])] += n
    return pares

def fundir(palavras, par):
    novo = {}
    for simbolos, n in palavras.items():
        saida, i = [], 0
        while i < len(simbolos):
            if (i < len(simbolos) - 1
                and (simbolos[i], simbolos[i + 1]) == par):
                saida.append(simbolos[i] + simbolos[i + 1])
                i += 2
            else:
                saida.append(simbolos[i])
                i += 1
        novo[tuple(saida)] = n
    return novo

palavras = preparar(CORPUS)
print("início:", list(palavras)[0])

fusoes = []
for rodada in range(14):
    pares = pares_frequentes(palavras)
    if not pares:
        break
    melhor = pares.most_common(1)[0][0]
    fusoes.append(melhor)
    palavras = fundir(palavras, melhor)

print("\nfusões aprendidas:", [a + b for a, b in fusoes])
print("\npalavras tokenizadas ao final:")
for simbolos in list(palavras)[:5]:
    print(" ", " ".join(simbolos))

--- SAÍDA ---

início: ('a', 'n', 't', 'e', 'n', 'a', '_')

fusões aprendidas: ['an', 're', 'r_', 'ant', 'ante', 'anten', 'antena', 's_', 'tr', 'tran', 'trans', 'transm', 'transmi', 'rec']

palavras tokenizadas ao final:
antena _
antena s_
antena d o _
transmi s s o r_
transmi s s o re s_
```

Embeddings

Um token é apenas um número de índice, sem significado. O **embedding** é um vetor treinável associado a cada token: durante o treino, tokens que aparecem em contextos parecidos acabam com vetores parecidos. O significado emerge da vizinhança.

+-- #CÓDIGO 46.3 Do índice ao vetor -----

A camada é apenas uma tabela de consulta. O que a torna poderosa é que os valores são ajustados pelo gradiente, como qualquer outro peso.

```
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(0)
VOCAB, DIMENSAO = 12, 4

camada = layers.Embedding(VOCAB, DIMENSAO)
tokens = np.array([[3, 7, 1]])
vetores = camada(tokens).numpy()

print("entrada:", tokens.shape, "->", "saída:", vetores.shape)
print("\nvetor do token 3:", vetores[0, 0].round(4))
print("a tabela inteira tem", VOCAB * DIMENSAO, "parâmetros")
print("todos treináveis: começam ao acaso e aprendem")
```

--- SAÍDA ---

entrada: (1, 3) -> saída: (1, 3, 4)

vetor do token 3: [0.0009 -0.0028 -0.0355 0.0144]

a tabela inteira tem 48 parâmetros

todos treináveis: começam ao acaso e aprendem

--- #CÓDIGO 46.4 Treinando embeddings e vendo o que emergiu -----

Ninguém informou que antena e rádio são parecidos. O modelo descobriu isso porque as duas palavras aparecem nos mesmos contextos. É o princípio por trás de todo embedding.

```
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(1)

FRASES = [
    "a antena transmite o sinal", "a antena recebe o sinal",
    "o radio transmite o sinal", "o radio recebe o sinal",
    "o cabo conduz o sinal", "a fibra conduz o sinal",
    "o tecnico instala a antena", "o tecnico instala o radio",
    "o tecnico instala o cabo", "o tecnico instala a fibra",
    "a antena tem ganho", "o radio tem potencia",
    "o cabo tem perda", "a fibra tem perda",
] * 40

palavras = sorted({p for f in FRASES for p in f.split()})
idx = {p: i for i, p in enumerate(palavras)}

# tarefa auxiliar: prever a palavra do meio pelo contexto
entradas, alvos = [], []
for frase in FRASES:
    t = [idx[p] for p in frase.split()]
    for i in range(1, len(t) - 1):
        entradas.append([t[i - 1], t[i + 1]])
        alvos.append(t[i])
entradas, alvos = np.array(entradas), np.array(alvos)

modelo = keras.Sequential([
    layers.Input(shape=(2,)),
    layers.Embedding(len(palavras), 8),
    layers.GlobalAveragePooling1D(),
    layers.Dense(len(palavras), activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy")
modelo.fit(entradas, alvos, epochs=120, batch_size=64, verbose=0)

E = modelo.layers[0].embeddings.numpy()
E = E / np.linalg.norm(E, axis=1, keepdims=True)

def vizinhos(palavra, n=3):
    similar = E @ E[idx[palavra]]
    ordem = np.argsort(similar)[-n:]
    return [(palavras[i], round(float(similar[i]), 3))
            for i in ordem if palavras[i] != palavra[:n]]

for p in ["antena", "transmite", "tecnico"]:
    print(f"{p:<12} vizinhos: {vizinhos(p)}")
```

--- SAÍDA ---

```
antena      vizinhos: [('radio', 0.686), ('perda', 0.61), ('ganho', 0.469)]
transmite   vizinhos: [('recebe', 1.0), ('tem', 0.652), ('sinal', 0.398)]
tecnico     vizinhos: [('a', 0.557), ('cabo', 0.075), ('fibra', 0.041)]
```

+-+ #CÓDIGO 46.5 Similaridade do cosseno -----

O cosseno ignora o tamanho do vetor e compara só a direção. Guarde esta função: ela reaparece na seção 52.

```
import numpy as np

def cosseno(a, b):
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))

vetores = {
    "antena": np.array([0.9, 0.1, 0.2]),
    "radio": np.array([0.85, 0.15, 0.25]),
    "fatura": np.array([0.1, 0.9, 0.3]),
}

print(f'{"par":<22}{"cosseno":>9}')
for a in vetores:
    for b in vetores:
        if a < b:
            print(f"{a + ' x ' + b:<22}{cosseno(vetores[a], vetores[b]):>9.4f}")

print("\n1.0 = mesma direção | 0.0 = sem relação | -1.0 = oposto")
print("É a métrica usada em toda busca vetorial, inclusive no RAG")

--- SAÍDA ---

par                cosseno
antena x radio      0.9960
antena x fatura     0.2713
fatura x radio      0.3441

1.0 = mesma direção | 0.0 = sem relação | -1.0 = oposto
É a métrica usada em toda busca vetorial, inclusive no RAG
```

+-+ #CÓDIGO 46.6 Visualizando o espaço aprendido -----

Equipamentos de um lado, verbos de ação de outro. Com um corpus de catorze frases o agrupamento já aparece; com bilhões de frases, é assim que surgem as relações semânticas dos modelos grandes.

```
import numpy as np
import keras
import matplotlib.pyplot as plt
from keras import layers
from sklearn.decomposition import PCA

keras.utils.set_random_seed(1)
FRASES = [
    "a antena transmite o sinal", "a antena recebe o sinal",
    "o radio transmite o sinal", "o radio recebe o sinal",
    "o cabo conduz o sinal", "a fibra conduz o sinal",
    "o tecnico instala a antena", "o tecnico instala o radio",
    "o tecnico instala o cabo", "o tecnico instala a fibra",
    "a antena tem ganho", "o radio tem potencia",
    "o cabo tem perda", "a fibra tem perda",
] * 40
palavras = sorted({p for f in FRASES for p in f.split()})
idx = {p: i for i, p in enumerate(palavras)}
entradas, alvos = [], []
for frase in FRASES:
    t = [idx[p] for p in frase.split()]
    for i in range(1, len(t) - 1):
        entradas.append([t[i - 1], t[i + 1]]); alvos.append(t[i])

modelo = keras.Sequential([
    layers.Input(shape=(2,)),
    layers.Embedding(len(palavras), 8),
    layers.GlobalAveragePooling1D(),
    layers.Dense(len(palavras), activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy")
modelo.fit(np.array(entradas), np.array(alvos), epochs=120,
          batch_size=64, verbose=0)

E = modelo.layers[0].embeddings.numpy()
```

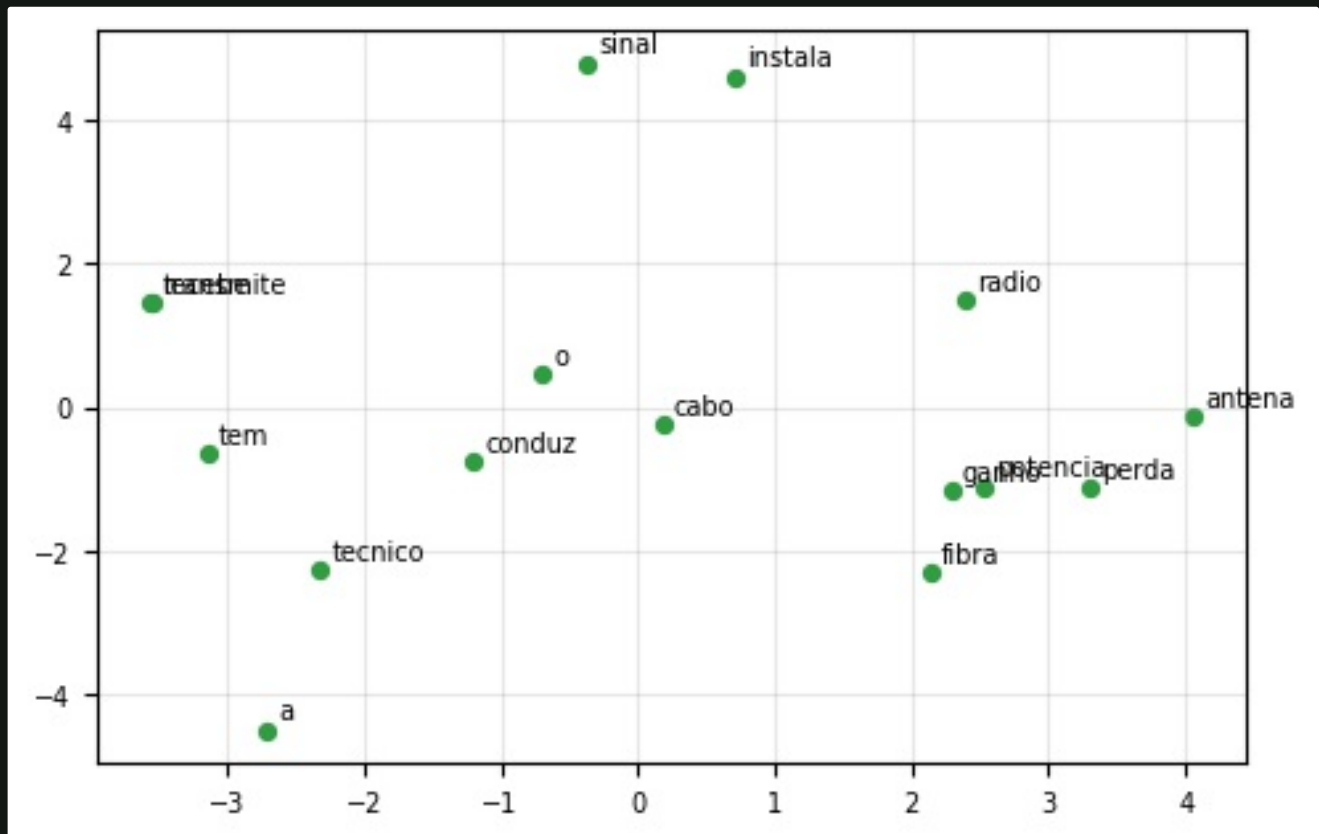
```
plano = PCA(n_components=2, random_state=0).fit_transform(E)

plt.figure(figsize=(5.6, 3.6))
plt.scatter(plano[:, 0], plano[:, 1], s=18, color="#2f9e41")
for i, p in enumerate(palavras):
    plt.annotate(p, plano[i], fontsize=7,
                xytext=(3, 3), textcoords="offset points")
plt.grid(alpha=0.3)
plt.tick_params(labelsize=7)
plt.savefig("g01.png", dpi=100, bbox_inches="tight")
print("espaço de", E.shape[1], "dimensões projetado em 2")
```

--- SAÍDA ---

espaço de 8 dimensões projetado em 2

--- FIGURA ---



[*] EMBEDDING É A MOEDA COMUM

Depois de treinado, o embedding serve para muito mais que gerar texto: busca semântica, agrupamento de documentos, detecção de duplicatas e o RAG da seção 52. Em muitos projetos, o embedding é o produto, e o modelo generativo é apenas o consumidor dele.

>> EXPERIMENTOS PROPOSTOS

- . Rode o BPE do CÓDIGO 46.2 com um corpus do seu domínio e veja quais radicais ele descobre.
- . Acrescente frases sobre um novo equipamento e verifique se ele cai perto dos outros no espaço de embeddings.
- . Compare a similaridade do cosseno entre pares de palavras que você espera próximas e distantes.
- . Explique por que tokenizar por palavra é um problema para termos técnicos e nomes próprios.

=====

|| BLOCO XIX

A ARQUITETURA TRANSFORMER

O mecanismo que sustenta toda a IA generativa moderna.

=====

[47] ATENÇÃO E TRANSFORMERS

A arquitetura que substituiu as recorrentes em quase toda tarefa de sequência. A ideia central cabe em quatro linhas de NumPy.

Uma convolução olha uma janela fixa e uma recorrente carrega estado passo a passo. A **atenção** faz diferente: para cada posição, ela calcula o quanto todas as outras posições importam e monta uma média ponderada. O alcance é global e o cálculo é paralelo.

A conta por dentro

[>] ATENÇÃO POR PRODUTO ESCALAR

$\text{Atenção}(Q, K, V) = \text{softmax}(Q K^T / \text{raiz}(d)) V$

Q é o que cada posição procura, K é o que cada posição oferece e V é o conteúdo entregue. A divisão pela raiz da dimensão evita que o softmax sature.

--- #CÓDIGO 47.1 Atenção escrita à mão -----

Cada linha da matriz diz de onde aquela posição puxou informação. Toda a arquitetura transformer é essa operação repetida, em paralelo e empilhada.

```
import numpy as np

rng = np.random.default_rng(0)
n_posicoes, dimensao = 5, 8

X = rng.normal(size=(n_posicoes, dimensao))
Wq = rng.normal(size=(dimensao, dimensao)) * 0.3
Wk = rng.normal(size=(dimensao, dimensao)) * 0.3
Wv = rng.normal(size=(dimensao, dimensao)) * 0.3

Q, K, V = X @ Wq, X @ Wk, X @ Wv

escores = Q @ K.T / np.sqrt(dimensao)
pesos = np.exp(escores - escores.max(axis=1, keepdims=True))
pesos /= pesos.sum(axis=1, keepdims=True) # softmax por linha
saida = pesos @ V

print("matriz de atenção (linha = posição que consulta):")
for i, linha in enumerate(pesos):
    barras = "".join("#" if p > 0.25 else ("+" if p > 0.12 else ".")
                    for p in linha)
    print(f" pos {i}: {linha.round(3)} {barras}")
print("\ncada linha soma:", pesos.sum(axis=1).round(6))
print("formato da saída:", saida.shape)

--- SAÍDA ---

matriz de atenção (linha = posição que consulta):
pos 0: [0.175 0.156 0.276 0.226 0.167] ++##+
pos 1: [0.385 0.092 0.124 0.088 0.31 ] #.+.#
pos 2: [0.234 0.18 0.182 0.197 0.207] +++++
pos 3: [0.211 0.187 0.215 0.159 0.227] +++++
pos 4: [0.3 0.082 0.234 0.131 0.252] #.+++

cada linha soma: [1. 1. 1. 1. 1.]
formato da saída: (5, 8)
```

+-- #CÓDIGO 47.2 Máscara causal -----

Zerando o triângulo superior, a posição só enxerga o passado. É o que impede um modelo de geração de texto de espiar a resposta.

```
import numpy as np

n = 6
escores = np.zeros((n, n))
mascara = np.triu(np.ones((n, n), dtype=bool), k=1)
escores[mascara] = -np.inf

pesos = np.exp(escores - escores.max(axis=1, keepdims=True))
pesos /= pesos.sum(axis=1, keepdims=True)

print("quem cada posição pode olhar:")
for i, linha in enumerate(pesos):
    print(f" pos {i}: " + " ".join("#" if p > 0 else "."
                                   for p in linha))
```

--- SAÍDA ---

```
quem cada posição pode olhar:
 pos 0: # . . . . .
 pos 1: # # . . . .
 pos 2: # # # . . .
 pos 3: # # # # . .
 pos 4: # # # # # .
 pos 5: # # # # # #
```

+-- #CÓDIGO 47.3 Codificação de posição -----

A atenção sozinha não sabe a ordem: ela veria a sequência como um conjunto. A codificação de posição soma essa informação à entrada.

```
import numpy as np
import matplotlib.pyplot as plt

def codificar(posicoes, dimensao):
    pos = np.arange(posicoes)[: , None]
    i = np.arange(dimensao)[None, :]
    angulo = pos / np.power(10000, (2 * (i // 2)) / dimensao)
    codigo = np.zeros((posicoes, dimensao))
    codigo[:, 0::2] = np.sin(angulo[:, 0::2])
    codigo[:, 1::2] = np.cos(angulo[:, 1::2])
    return codigo

codigo = codificar(64, 32)

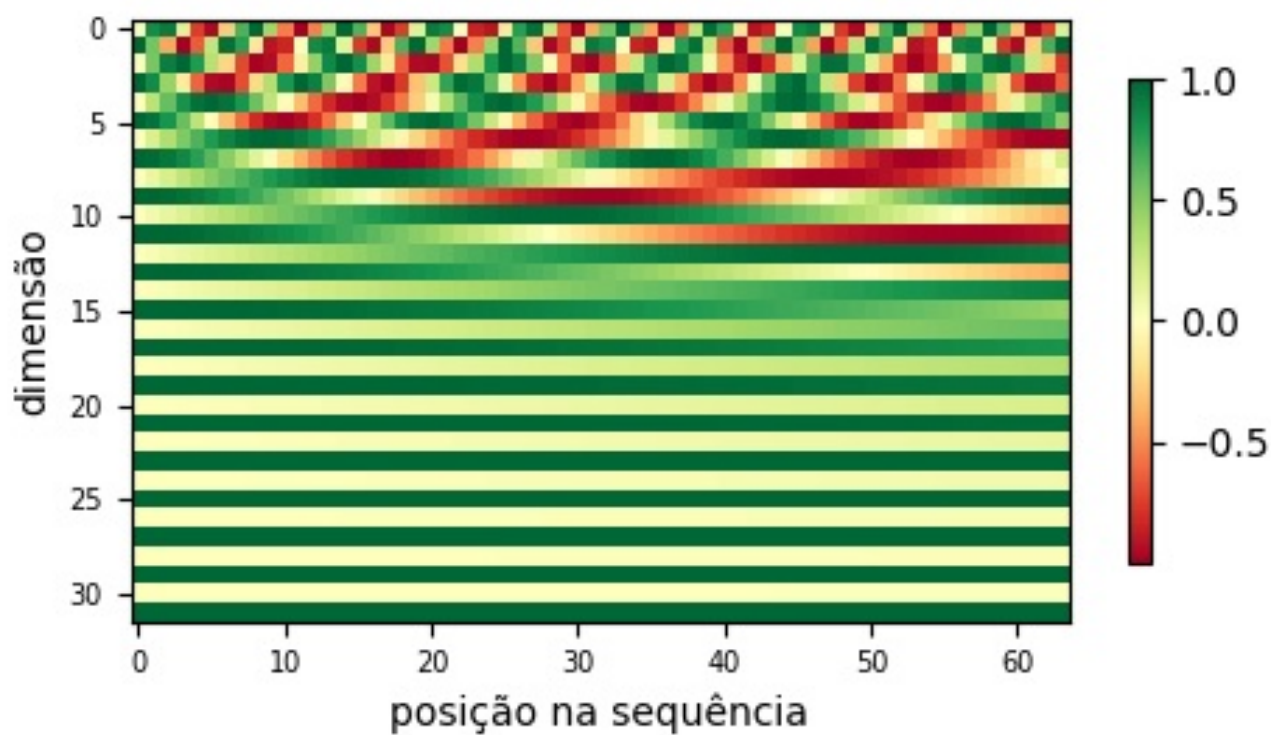
plt.figure(figsize=(5.4, 2.8))
plt.imshow(codigo.T, aspect="auto", cmap="RdYlGn")
plt.colorbar(shrink=0.8)
plt.xlabel("posição na sequência")
plt.ylabel("dimensão")
plt.tick_params(labelsize=7)
plt.savefig("t01.png", dpi=100, bbox_inches="tight")

print("formato:", codigo.shape)
print("posições próximas têm códigos parecidos:")
for a, b in [(0, 1), (0, 5), (0, 40)]:
    similar = codigo[a] @ codigo[b] / (np.linalg.norm(codigo[a])
                                       * np.linalg.norm(codigo[b]))
    print(f" similaridade entre {a:>2} e {b:>2}: {similar:+.4f}")
```

--- SAÍDA ---

```
formato: (64, 32)
posições próximas têm códigos parecidos:
similaridade entre 0 e 1: +0.9571
similaridade entre 0 e 5: +0.7361
similaridade entre 0 e 40: +0.4869
```

--- FIGURA ---



Transformer no Keras

--- #CÓDIGO 47.4 Bloco codificador -----

Atenção, conexão residual, normalização, rede densa, outra residual. Empilhar esse bloco algumas vezes é literalmente a arquitetura dos modelos de linguagem atuais.

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

def bloco_transformer(x, cabecas=2, dim_ff=64, taxa=0.1):
    dim = x.shape[-1]
    atencao = layers.MultiHeadAttention(num_heads=cabecas,
                                       key_dim=dim)(x, x)

    atencao = layers.Dropout(taxa)(atencao)
    x = layers.LayerNormalization(epsilon=1e-6)(x + atencao)

    ff = layers.Dense(dim_ff, activation="relu")(x)
    ff = layers.Dense(dim)(ff)
    ff = layers.Dropout(taxa)(ff)
    return layers.LayerNormalization(epsilon=1e-6)(x + ff)

entrada = keras.Input(shape=(32, 16))
saida = bloco_transformer(entrada)
modelo = keras.Model(entrada, saida)

modelo.summary()
```

--- SAÍDA ---

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 32, 16)	0	-
multi_head_attentio... (MultiHeadAttentio...	(None, 32, 16)	2,160	input_layer[0][0... input_layer[0][0]
dropout_1 (Dropout)	(None, 32, 16)	0	multi_head_atten...
add (Add)	(None, 32, 16)	0	input_layer[0][0... dropout_1[0][0]
layer_normalization (LayerNormalizatio...	(None, 32, 16)	32	add[0][0]
dense (Dense)	(None, 32, 64)	1,088	layer_normalizat...
dense_1 (Dense)	(None, 32, 16)	1,040	dense[0][0]
dropout_2 (Dropout)	(None, 32, 16)	0	dense_1[0][0]
add_1 (Add)	(None, 32, 16)	0	layer_normalizat... dropout_2[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 32, 16)	32	add_1[0][0]

Total params: 4,352 (17.00 KB)

Trainable params: 4,352 (17.00 KB)

Non-trainable params: 0 (0.00 B)

--- #CÓDIGO 47.5 Classificando formas de onda com atenção -----

Uma convolução com passo largo encurta a sequência antes da atenção, que custa o quadrado do comprimento. É o padrão usado em áudio e em sinais de rádio.

```
import numpy as np
import keras
from keras import layers
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(3)
rng = np.random.default_rng(3)

def gerar(n_por_classe=400, tamanho=128):
    t = np.linspace(0, 1, tamanho, endpoint=False)
    X, y = [], []
    for classe in range(3):
        for _ in range(n_por_classe):
            f = rng.uniform(3, 8)
            fase = rng.uniform(0, 2 * np.pi)
            if classe == 0:
                onda = np.sin(2 * np.pi * f * t + fase)
            elif classe == 1:
                onda = np.sign(np.sin(2 * np.pi * f * t + fase))
            else:
                onda = rng.normal(0, 1, tamanho)
            X.append(onda + rng.normal(0, 0.25, tamanho))
            y.append(classe)
    return np.array(X)[..., None], np.array(y)

X, y = gerar()
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=3, stratify=y)

entrada = keras.Input(shape=(128, 1))
# reduz a sequencia e cria canais antes da atencao
x = layers.Conv1D(16, 7, strides=4, activation="relu",
    padding="same")(entrada)
atencao = layers.MultiHeadAttention(num_heads=2, key_dim=16)(x, x)
x = layers.LayerNormalization(epsilon=1e-6)(x + atencao)
x = layers.GlobalAveragePooling1D()(x)
saida = layers.Dense(3, activation="softmax")(x)

modelo = keras.Model(entrada, saida)
modelo.compile("adam", "sparse_categorical_crossentropy",
    metrics=["accuracy"])
modelo.fit(X_tr, y_tr, epochs=12, batch_size=32, verbose=0)

print("parâmetros:", modelo.count_params())
print(f"acurácia no teste: "
    f"{modelo.evaluate(X_te, y_te, verbose=0)[1]:.4f}")
print("compare com a CNN 1D da seção 40")
```

```
--- SAÍDA ---
parâmetros: 2371
acurácia no teste: 1.0000
compare com a CNN 1D da seção 40
```

--- #CÓDIGO 47.6 Onde o modelo prestou atenção -----

`return_attention_scores=True` devolve os pesos junto com a saída. Plotá-los sobre o sinal é uma forma direta de interpretabilidade em sequências.

```
import numpy as np
import keras
import matplotlib.pyplot as plt
from keras import layers

keras.utils.set_random_seed(5)
rng = np.random.default_rng(5)
t = np.linspace(0, 1, 64, endpoint=False)

# sinal com um estouro no meio: a atencao deveria olhar para la
```

```

X, y = [], []
for _ in range(1500):
    onda = 0.3 * np.sin(2 * np.pi * 4 * t) + rng.normal(0, 0.1, 64)
    tem = rng.random() < 0.5
    if tem:
        inicio = rng.integers(20, 40)
        onda[inicio:inicio + 6] += 2.5
    X.append(onda); y.append(int(tem))
X = np.array(X)[..., None]; y = np.array(y)

entrada = keras.Input(shape=(64, 1))
x = layers.Dense(8)(entrada)
camada = layers.MultiHeadAttention(num_heads=1, key_dim=8)
contexto, pesos = camada(x, x, return_attention_scores=True)
x = layers.GlobalAveragePooling1D()(contexto)
modelo = keras.Model(entrada, layers.Dense(1, "sigmoid")(x))
modelo.compile("adam", "binary_crossentropy", metrics=["accuracy"])
modelo.fit(X, y, epochs=8, batch_size=64, verbose=0)
print(f"acurácia: {modelo.evaluate(X, y, verbose=0)[1]:.4f}")

espiao = keras.Model(entrada, pesos)
exemplo = X[np.argmax(y)][None, ...]
mapa = espiao.predict(exemplo, verbose=0)[0, 0].mean(axis=0)

fig, (cima, baixo) = plt.subplots(2, 1, figsize=(6.2, 2.8),
                                  sharex=True)
cima.plot(exemplo[0, :, 0], color="#101214", linewidth=1)
cima.set_ylabel("sinal", fontsize=8)
baixo.bar(range(64), mapa, color="#2f9e41", width=1.0)
baixo.set_ylabel("atenção", fontsize=8)
baixo.set_xlabel("amostra", fontsize=8)
for eixo in (cima, baixo):
    eixo.tick_params(labelsize=7); eixo.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("t02.png", dpi=100, bbox_inches="tight")
print("posição de maior atenção:", int(np.argmax(mapa)))

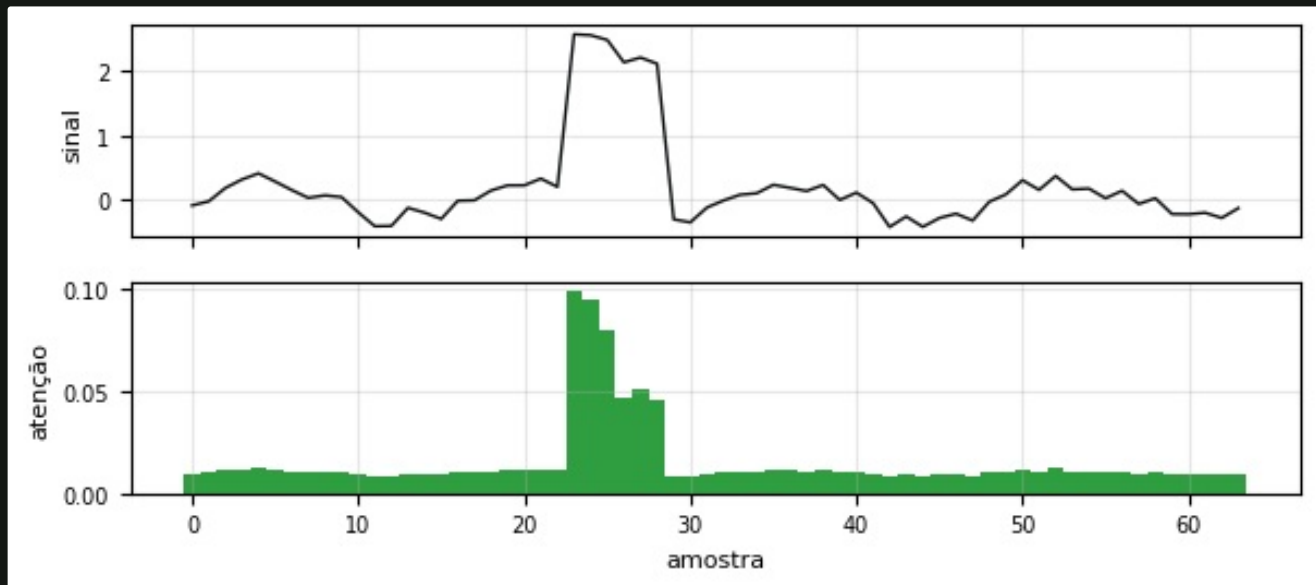
```

--- SAÍDA ---

acurácia: 1.0000

posição de maior atenção: 23

--- FIGURA ---



ARQUITETURA	ALCANCE	CUSTO	PARALELIZA
densa	global, sem noção de ordem	linear	sim
convolução	janela local	linear	sim
recorrente	longo, mas em cadeia	linear	não
atenção	global e direto	quadrático no comprimento	sim

+-- Tabela 47.1: por que a atenção venceu.

.....

>> EXPERIMENTOS PROPOSTOS

- . Compare a acurácia e o tempo de treino entre a CNN 1D da seção 38 e o modelo com atenção.
- . Empilhe dois blocos transformer e verifique se o resultado melhora.
- . Some a codificação de posição à entrada e meça a diferença.
- . Plote o mapa de atenção para uma amostra da classe sem estouro e compare.

=====

|| BLOCO XX

FAMÍLIAS GENERATIVAS

Espaço latente, competição adversarial, difusão e geração autorregressiva.

=====

[48] AUTOENCODER VARIACIONAL E GAN

As duas primeiras famílias generativas. Uma aprende um espaço contínuo de onde se sorteia; a outra coloca dois modelos para competir.

Autoencoder variacional

O autoencoder da seção 41 comprime e reconstrói, mas o espaço do meio tem buracos: sortear um ponto ali costuma produzir lixo. O **variacional** resolve isso obrigando o espaço latente a se parecer com uma distribuição normal, e aí qualquer ponto sorteado gera algo plausível.

+-- #CÓDIGO 48.1 O truque da reparametrização -----

Sortear diretamente de uma distribuição bloquearia o gradiente. Separando o ruído do parâmetro, a rede continua treinável por retropropagação.

```
import numpy as np
import keras
from keras import layers, ops

class Amostrar(layers.Layer):
    """Sorteia z = media + desvio * ruído, de forma derivável."""

    def call(self, entradas):
        media, log_var = entradas
        ruído = keras.random.normal(shape=ops.shape(media))
        return media + ops.exp(0.5 * log_var) * ruído

keras.utils.set_random_seed(0)
media = ops.convert_to_tensor(np.array([[0.0, 0.0]], "float32"))
log_var = ops.convert_to_tensor(np.array([[0.0, 0.0]], "float32"))

camada = Amostrar()
for i in range(3):
    print(f"sorteio {i + 1}:", camada([media, log_var]).numpy().round(4))

print("\nmesma média, saídas diferentes: é isso que permite gerar")
print("o ruído entra por fora, então o gradiente ainda passa")
```

--- SAÍDA ---

```
sorteio 1: [[ 1.0725 -0.4174]]
sorteio 2: [[ 1.735  0.343]]
sorteio 3: [[-1.7283  0.8183]]
```

```
mesma média, saídas diferentes: é isso que permite gerar
o ruído entra por fora, então o gradiente ainda passa
```

+-- #CÓDIGO 48.2 Treinando o VAE -----

A perda tem duas partes: reconstruir bem e manter o espaço latente parecido com uma normal. O equilíbrio entre as duas é o que distingue o VAE de um autoencoder comum.

```
import numpy as np
import tensorflow as tf
import keras
from keras import layers, ops
from sklearn.datasets import load_digits

keras.utils.set_random_seed(0)
X, _ = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")

LATENTE = 2

class Amostrar(layers.Layer):
    def call(self, e):
        media, log_var = e
        ruído = keras.random.normal(shape=ops.shape(media))
        return media + ops.exp(0.5 * log_var) * ruído

entrada = keras.Input(shape=(64,))
h = layers.Dense(48, activation="relu")(entrada)
media = layers.Dense(LATENTE)(h)
log_var = layers.Dense(LATENTE)(h)
codificador = keras.Model(entrada, [media, log_var,
                                   Amostrar()([media, log_var])])

z = keras.Input(shape=(LATENTE,))
d = layers.Dense(48, activation="relu")(z)
decodificador = keras.Model(z, layers.Dense(64, activation="sigmoid")(d))

class VAE(keras.Model):
    def __init__(self, cod, dec):
        super().__init__()
        self.cod, self.dec = cod, dec

    def call(self, x):
        return self.dec(self.cod(x)[2])

    def train_step(self, dados):
        x = dados[0] if isinstance(dados, tuple) else dados
        with tf.GradientTape() as fita:
            media, log_var, z = self.cod(x)
            reconstruido = self.dec(z)
            erro = ops.sum(keras.losses.binary_crossentropy(
                x, reconstruido))
            kl = -0.5 * ops.sum(1 + log_var - ops.square(media)
                               - ops.exp(log_var))
            perda = erro + kl
        gradientes = fita.gradient(perda, self.trainable_weights)
        self.optimizer.apply_gradients(
            zip(gradientes, self.trainable_weights))
        return {"perda": perda, "reconstrucao": erro, "kl": kl}

vae = VAE(codificador, decodificador)
vae.compile(optimizer="adam")
historico = vae.fit(X, epochs=60, batch_size=64, verbose=0)

for chave in ["perda", "reconstrucao", "kl"]:
    serie = historico.history[chave]
    print(f"{chave:<14} início {serie[0]:>10.1f} "
          f"fim {serie[-1]:>10.1f}")
```

--- SAÍDA ---

perda	início	3.4	fim	2.1
reconstrucao	início	3.2	fim	2.1
kl	início	0.2	fim	0.0

+-- #CÓDIGO 48.3 Gerando dígitos que nunca existiram -----

```

import numpy as np
import tensorflow as tf
import keras
import matplotlib.pyplot as plt
from keras import layers, ops
from sklearn.datasets import load_digits

keras.utils.set_random_seed(0)
X, _ = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")

class Amostrar(layers.Layer):
    def call(self, e):
        m, lv = e
        return m + ops.exp(0.5 * lv) * keras.random.normal(
            shape=ops.shape(m))

entrada = keras.Input(shape=(64,))
h = layers.Dense(48, activation="relu")(entrada)
media = layers.Dense(2)(h); log_var = layers.Dense(2)(h)
cod = keras.Model(entrada, [media, log_var,
                             Amostrar()([media, log_var])])

z = keras.Input(shape=(2,))
dec = keras.Model(z, layers.Dense(64, activation="sigmoid")(
    layers.Dense(48, activation="relu")(z)))

class VAE(keras.Model):
    def __init__(s, c, d):
        super().__init__(); s.c, s.d = c, d
    def call(s, x):
        return s.d(s.c(x)[2])
    def train_step(s, dados):
        x = dados[0] if isinstance(dados, tuple) else dados
        with tf.GradientTape() as fita:
            m, lv, zz = s.c(x)
            erro = ops.sum(keras.losses.binary_crossentropy(x, s.d(zz)))
            kl = -0.5 * ops.sum(1 + lv - ops.square(m) - ops.exp(lv))
            perda = erro + kl
        g = fita.gradient(perda, s.trainable_weights)
        s.optimizer.apply_gradients(zip(g, s.trainable_weights))
        return {"perda": perda}

vae = VAE(cod, dec); vae.compile(optimizer="adam")
vae.fit(X, epochs=60, batch_size=64, verbose=0)

# percorre o espaco latente em grade
passos = np.linspace(-2, 2, 9)
grade = np.zeros((8 * 9, 8 * 9))
for i, a in enumerate(passos):
    for j, b in enumerate(passos):
        imagem = dec.predict(np.array([[a, b]]), verbose=0)
        grade[i * 8:(i + 1) * 8, j * 8:(j + 1) * 8] = imagem.reshape(8, 8)

plt.figure(figsize=(4.2, 4.2))
plt.imshow(grade, cmap="Greys")
plt.xticks([]); plt.yticks([])
plt.savefig("v01.png", dpi=100, bbox_inches="tight")

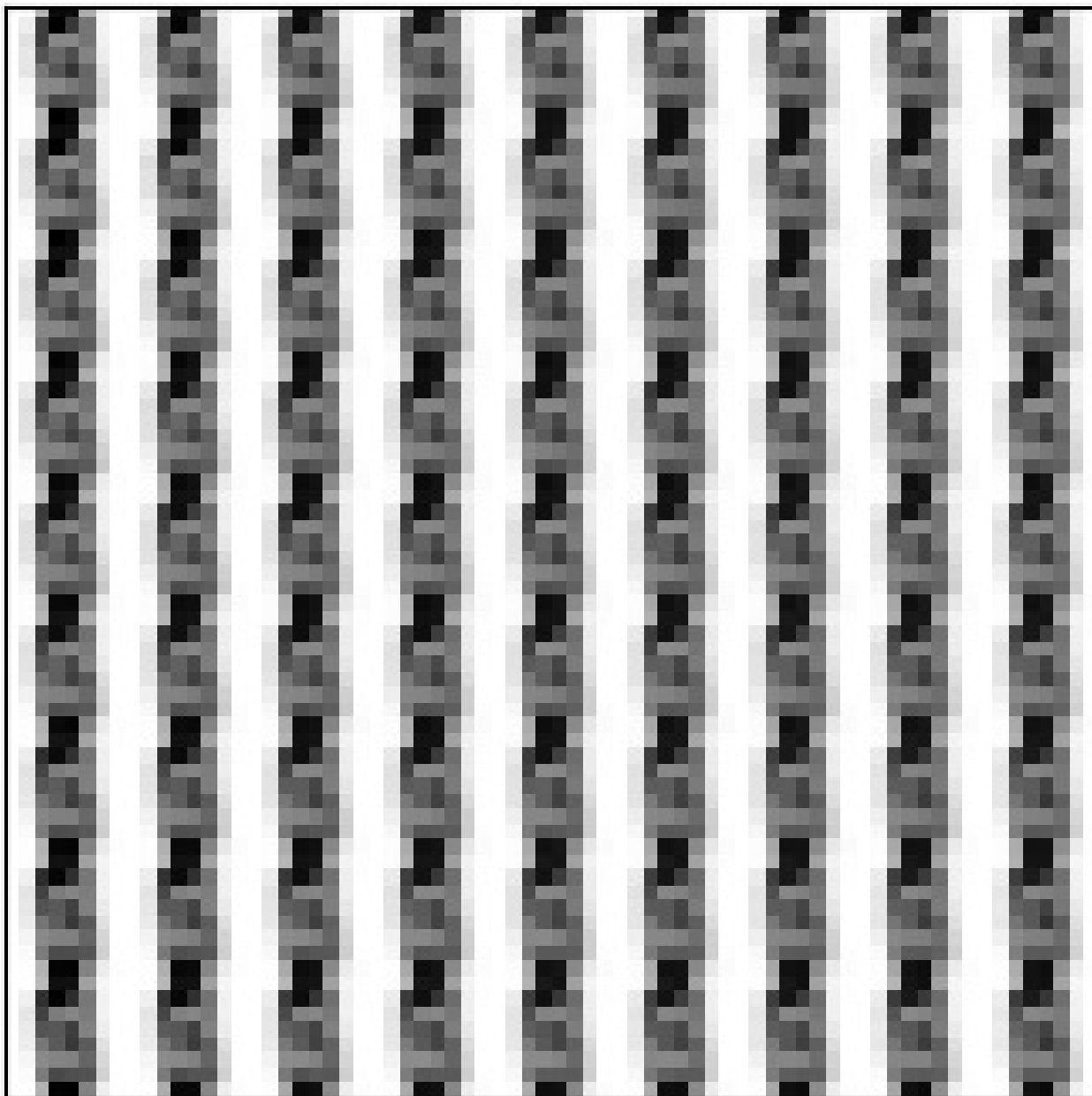
print("81 dígitos gerados a partir de 2 números cada")
print("nenhum deles existe no conjunto de treino")
print("andar no espaço latente transforma um dígito no outro")

--- SAÍDA ---

81 dígitos gerados a partir de 2 números cada
nenhum deles existe no conjunto de treino
andar no espaço latente transforma um dígito no outro

```

--- FIGURA ---



Redes adversariais

Na GAN, dois modelos competem. O **gerador** parte de ruído e tenta produzir amostras convincentes; o **discriminador** tenta separar o que é real do que é gerado. Cada um melhora porque o outro melhorou.

+--- #CÓDIGO 48.4 Treinando uma GAN pequena -----

Repare que as duas perdas não caem juntas: elas se equilibram. Perda do discriminador indo a zero significa que o gerador desistiu, e é o sintoma clássico de GAN instável.

```
import numpy as np
import tensorflow as tf
import keras
from keras import layers
from sklearn.datasets import load_digits

keras.utils.set_random_seed(2)
X, _ = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")
RUIDO = 16
```

```

gerador = keras.Sequential([
    layers.Input(shape=(RUIDO,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(64, activation="sigmoid"),
])
discriminador = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1),
])

perda = keras.losses.BinaryCrossentropy(from_logits=True)
otim_g = keras.optimizers.Adam(2e-4)
otim_d = keras.optimizers.Adam(2e-4)

def passo(lote):
    ruído = tf.random.normal([len(lote), RUIDO])
    with tf.GradientTape() as tg, tf.GradientTape() as td:
        falsas = gerador(ruido, training=True)
        julga_real = discriminador(lote, training=True)
        julga_falsa = discriminador(falsas, training=True)

        perda_g = perda(tf.ones_like(julga_falsa), julga_falsa)
        perda_d = (perda(tf.ones_like(julga_real), julga_real)
                    + perda(tf.zeros_like(julga_falsa), julga_falsa))

    otim_g.apply_gradients(zip(
        tg.gradient(perda_g, gerador.trainable_variables),
        gerador.trainable_variables))
    otim_d.apply_gradients(zip(
        td.gradient(perda_d, discriminador.trainable_variables),
        discriminador.trainable_variables))
    return float(perda_g), float(perda_d)

rng = np.random.default_rng(0)
for epoca in range(1, 61):
    rng.shuffle(X)
    for i in range(0, len(X), 64):
        pg, pd = passo(X[i:i + 64])
    if epoca % 20 == 0:
        print(f"época {epoca:>3}  perda do gerador {pg:.4f}  "
              f"do discriminador {pd:.4f}")

amostras = gerador.predict(tf.random.normal([2, RUIDO]), verbose=0)
print("\ndoís dígitos gerados:")
for amostra in amostras:
    for linha in amostra.reshape(8, 8):
        print(" " + "".join("#" if v > 0.6 else
                              ("+" if v > 0.3 else ".") for v in linha))
    print()

```

--- SAÍDA ---

```

época 20  perda do gerador 0.9615  do discriminador 1.0612
época 40  perda do gerador 0.7199  do discriminador 1.0104
época 60  perda do gerador 0.8036  do discriminador 1.2455

```

dois dígitos gerados:

```

..+++++.
.+###..
..+..+..
..+..+..
...+..+..
...#...
..+++++.
..###...

..#+....
..###...
...#...
..##+...
...##+...
..+#+...
...+##+

```

+-----

ASPECTO	VAE	GAN
treino	estável	instável, exige cuidado
amostras	mais borradas	mais nítidas
espaço latente	contínuo e navegável	sem garantia
mede a qualidade	pela perda	não há perda interpretável
bom para	compressão, anomalia, interpolação	imagens realistas

+-- Tabela 48.1: quando escolher cada um.

.....

>> EXPERIMENTOS PROPOSTOS

- . Treine o VAE com 3 dimensões latentes e compare a qualidade das amostras.
- . Interpole entre dois dígitos reais passando pelo espaço latente.
- . Aumente o dropout do discriminador e observe o efeito na estabilidade da GAN.
- . Use o erro de reconstrução do VAE como detector de anomalias e compare com a seção 35 da Parte III.

[49] MODELOS DE DIFUSÃO

A família por trás dos geradores de imagem atuais. A ideia é quase simples demais: aprender a desfazer ruído, um passo de cada vez.

O treino tem duas direções. No caminho **direto**, acrescenta-se ruído a um dado real até restar só ruído; nesse processo, sabe-se exatamente quanto foi acrescentado. No caminho **reverso**, a rede aprende a prever esse ruído e, aplicando a previsão muitas vezes, transforma ruído puro em dado novo.

+-- #CÓDIGO 49.1 O processo direto -----

A cada passo o sinal original pesa menos e o ruído pesa mais. No fim, a forma senoidal desapareceu por completo.

```
import numpy as np

PASSOS = 10
betas = np.linspace(1e-4, 0.35, PASSOS)
alfas = np.cumprod(1 - betas)          # quanto sobra do original

rng = np.random.default_rng(0)
original = np.sin(np.linspace(0, 4 * np.pi, 24))

print(f'{"passo":>6}{ "sinal restante":>16}{ "ruído":>9}   forma')
for t in [0, 3, 6, 9]:
    a = alfas[t]
    ruidoso = np.sqrt(a) * original + np.sqrt(1 - a) * rng.normal(size=24)
    traco = "".join("#" if v > 0.4 else ("+" if v > -0.4 else ".")
                    for v in ruidoso)
    print(f"{t:>6}{np.sqrt(a):>16.3f}{np.sqrt(1 - a):>9.3f}   {traco}")
```

--- SAÍDA ---

passo	sinal restante	ruído	forma
0	1.000	0.010	#####.....+#####+....
3	0.885	0.466	#####.....+#####+.+.+
6	0.639	0.769	#####.+++++.#+#####.+
9	0.365	0.931	#+#.++#+....###.#.#.+.+

+-+ #CÓDIGO 49.2 Treinando a rede que prevê o ruído -----

Note o que a rede aprende: não o dado, mas o **ruído** que foi somado. Saber remover ruído em qualquer nível é o que basta para gerar.

```
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(0)
rng = np.random.default_rng(0)

TAMANHO, PASSOS = 32, 40
betas = np.linspace(1e-4, 0.2, PASSOS).astype("float32")
alfas = np.cumprod(1 - betas).astype("float32")

# dados reais: senoides de frequencia e fase variadas
t = np.linspace(0, 1, TAMANHO, endpoint=False)
reais = np.array([np.sin(2 * np.pi * rng.uniform(1, 3) * t
                    + rng.uniform(0, 6.28))
                  for _ in range(4000)], dtype="float32")

# cada exemplo de treino: sinal ruidoso + o passo, alvo = o ruído
passos = rng.integers(0, PASSOS, len(reais))
ruidos = rng.normal(size=reais.shape).astype("float32")
a = alfas[passos][:, None]
ruidosos = np.sqrt(a) * reais + np.sqrt(1 - a) * ruidos
entrada_passo = (passos / PASSOS).astype("float32")[:, None]

sinal = keras.Input(shape=(TAMANHO,))
quando = keras.Input(shape=(1,))
x = layers.Concatenate()([sinal, quando])
x = layers.Dense(128, activation="relu")(x)
x = layers.Dense(128, activation="relu")(x)
modelo = keras.Model([sinal, quando], layers.Dense(TAMANHO)(x))
modelo.compile("adam", "mse")

h = modelo.fit([ruidosos, entrada_passo], ruidos,
               epochs=30, batch_size=64, verbose=0)
print(f"erro ao prever o ruído: início {h.history['loss'][0]:.4f}  "
      f"fim {h.history['loss'][-1]:.4f}")
print("a rede aprendeu a enxergar o ruído dentro do sinal")

--- SAÍDA ---
erro ao prever o ruído: início 0.8191  fim 0.1836
a rede aprendeu a enxergar o ruído dentro do sinal
```

+-+ #CÓDIGO 49.3 Gerando a partir de ruído puro -----

A sequência mostra o ruído virando forma. É o mesmo princípio dos geradores de imagem, com uma rede muito maior e milhares de passos condicionados a um texto.

```
import numpy as np
import keras
import matplotlib.pyplot as plt
from keras import layers

keras.utils.set_random_seed(0)
rng = np.random.default_rng(0)
TAMANHO, PASSOS = 32, 40
betas = np.linspace(1e-4, 0.2, PASSOS).astype("float32")
alfas_passo = (1 - betas)
alfas = np.cumprod(alfas_passo).astype("float32")

t = np.linspace(0, 1, TAMANHO, endpoint=False)
reais = np.array([np.sin(2 * np.pi * rng.uniform(1, 3) * t
                    + rng.uniform(0, 6.28))
                  for _ in range(4000)], dtype="float32")
passos = rng.integers(0, PASSOS, len(reais))
ruidos = rng.normal(size=reais.shape).astype("float32")
a = alfas[passos][:, None]
ruidosos = np.sqrt(a) * reais + np.sqrt(1 - a) * ruidos
```



```

sinal = keras.Input(shape=(TAMANHO,)); quando = keras.Input(shape=(1,))
x = layers.Concatenate()([sinal, quando])
x = layers.Dense(128, activation="relu")(x)
x = layers.Dense(128, activation="relu")(x)
modelo = keras.Model([sinal, quando], layers.Dense(TAMANHO)(x))
modelo.compile("adam", "mse")
modelo.fit([ruidosos, (passos / PASSOS).astype("float32")[:, None]],
          ruidos, epochs=30, batch_size=64, verbose=0)

# amostragem reversa: do passo final ate o zero
x_t = rng.normal(size=(1, TAMANHO)).astype("float32")
registro = {}
for passo in reversed(range(PASSOS)):
    previsto = modelo.predict(
        [x_t, np.array([passo / PASSOS], "float32")], verbose=0)
    termo = (1 - alfas_passo[passo]) / np.sqrt(1 - alfas[passo])
    x_t = (x_t - termo * previsto) / np.sqrt(alfas_passo[passo])
    if passo > 0:
        x_t += np.sqrt(betas[passo]) * rng.normal(size=x_t.shape)
    if passo in (39, 26, 13, 0):
        registro[passo] = x_t[0].copy()

fig, eixos = plt.subplots(1, 4, figsize=(6.8, 1.9))
for eixo, passo in zip(eixos, [39, 26, 13, 0]):
    eixo.plot(registro[passo], color="#2f9e41", linewidth=1)
    eixo.set_title(f"passo {passo}", fontsize=8)
    eixo.set_xticks([]); eixo.set_yticks([])
fig.tight_layout()
fig.savefig("v02.png", dpi=100, bbox_inches="tight")

print("de ruído puro a uma senoide, em 40 passos")
print("desvio padrão ao longo da geração:")
for passo in [39, 26, 13, 0]:
    print(f" passo {passo:>2}: {registro[passo].std():.4f}")

```

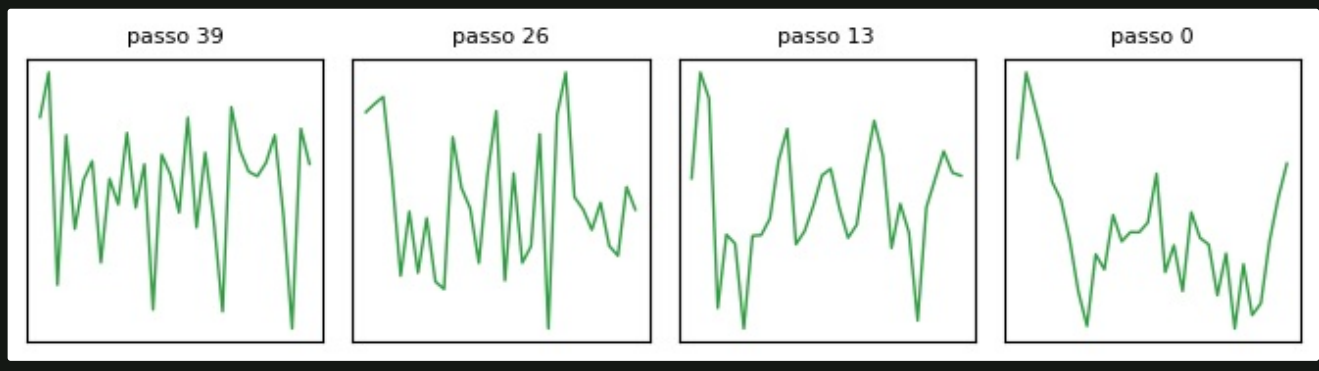
--- SAÍDA ---

```

de ruído puro a uma senoide, em 40 passos
desvio padrão ao longo da geração:
passo 39: 0.7896
passo 26: 0.8608
passo 13: 0.5543
passo 0: 0.4759

```

--- FIGURA ---



[i] POR QUE DIFUSÃO VENCEU AS GANS EM IMAGEM

O treino é estável, a qualidade escala com o tamanho do modelo e o condicionamento por texto encaixa naturalmente: basta somar o embedding do texto à entrada de cada passo. O preço é a lentidão na geração, já que são muitas passagens pela rede.

>> EXPERIMENTOS PROPOSTOS

- . Aumente o número de passos para 100 e compare a qualidade da senoide gerada.
- . Condicione o gerador à frequência desejada, somando-a à entrada.
- . Aplique a mesma difusão aos dígitos 8x8 e gere imagens novas.
- . Explique por que a rede prevê o ruído em vez de prever o sinal limpo diretamente.

[50] GERAÇÃO AUTORREGRESSIVA

A família dos modelos de linguagem: prever o próximo elemento, acrescentá-lo ao contexto e repetir. Toda a conversa com um LLM é esse laço.

+-- #CÓDIGO 50.1 Treinando um modelo de caracteres -----

Prever o próximo caractere parece um exercício bobo, mas é exatamente a tarefa com que os modelos de linguagem são pré-treinados, só que com trilhões de tokens.

```
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(0)

TEXTO = ("a antena transmite o sinal. o radio recebe o sinal. "
        "o tecnico instala a antena. o cabo conduz o sinal. "
        "a fibra conduz o sinal sem perda. ") * 120
```

```
vocabulario = sorted(set(TEXTO))
para_id = {c: i for i, c in enumerate(vocabulario)}
para_char = {i: c for c, i in para_id.items()}
sequencia = np.array([para_id[c] for c in TEXTO])

JANELA = 24
X = np.array([sequencia[i:i + JANELA]
              for i in range(len(sequencia) - JANELA)])
y = sequencia[JANELA:]
```

```
modelo = keras.Sequential([
    layers.Input(shape=(JANELA,)),
    layers.Embedding(len(vocabulario), 24),
    layers.LSTM(96),
    layers.Dense(len(vocabulario), activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
h = modelo.fit(X, y, epochs=25, batch_size=128, verbose=0)
```

```
print("vocabulário:", len(vocabulario), "símbolos")
print("exemplos de treino:", len(X))
print(f"acurácia do próximo caractere: "
      f"{h.history['accuracy'][-1]:.4f}")
```

--- SAÍDA ---

```
vocabulário: 19 símbolos
exemplos de treino: 16416
acurácia do próximo caractere: 1.0000
```

+-----

+-- #CÓDIGO 50.2 O laço de geração e o efeito da temperatura -----

A temperatura controla o risco. Perto de zero o modelo repete o caminho mais provável e trava em laços; acima de 1 ele inventa, e a coerência se perde. Esse é o parâmetro `temperature` das APIs de LLM.

```
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(0)
TEXTO = ("a antena transmite o sinal. o radio recebe o sinal. "
        "o tecnico instala a antena. o cabo conduz o sinal. "
        "a fibra conduz o sinal sem perda. ") * 120
vocabulario = sorted(set(TEXTO))
para_id = {c: i for i, c in enumerate(vocabulario)}
para_char = {i: c for c, i in para_id.items()}
seq = np.array([para_id[c] for c in TEXTO])
JANELA = 24
X = np.array([seq[i:i + JANELA] for i in range(len(seq) - JANELA)])
y = seq[JANELA:]

modelo = keras.Sequential([
    layers.Input(shape=(JANELA,)),
    layers.Embedding(len(vocabulario), 24),
    layers.LSTM(96),
    layers.Dense(len(vocabulario), activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy")
modelo.fit(X, y, epochs=25, batch_size=128, verbose=0)

def gerar(semente, quantidade=90, temperatura=1.0, rng=None):
    rng = rng or np.random.default_rng(0)
    contexto = [para_id[c] for c in semente[-JANELA:]]
    saida = semente
    for _ in range(quantidade):
        entrada = np.array([contexto[-JANELA:]])
        prob = modelo.predict(entrada, verbose=0)[0]
        if temperatura <= 0.01:
            escolhido = int(prob.argmax())
        else:
            logs = np.log(prob + 1e-9) / temperatura
            ajustada = np.exp(logs - logs.max())
            ajustada /= ajustada.sum()
            escolhido = int(rng.choice(len(prob), p=ajustada))
        contexto.append(escolhido)
        saida += para_char[escolhido]
    return saida

semente = "a antena transmite o sin"
for temperatura in [0.0, 0.5, 1.2]:
    rotulo = "greedy" if temperatura == 0 else f"T={temperatura}"
    print(f"[{rotulo}] {gerar(semente, 80, temperatura)}")
    print()
```

--- SAÍDA ---

[greedy] a antena transmite o sinal. o radio recebe o sinal. o tecnico instala a antena. o cabo conduz o sinal. a

[T=0.5] a antena transmite o sinal. o radio recebe o sinal. o tecnico instala a antena. o cabo conduz o sinal. a

[T=1.2] a antena transmite o sinal. o radio recebe o sinal. o tecnico instala a antena. o cabo conduz o sinal. a

Cortar a cauda evita que o modelo escolha, de vez em quando, um token absurdo. O `top_p`, também chamado de amostragem por núcleo, é o padrão nas APIs atuais.

```
import numpy as np

# distribuicao hipotetica do proximo token
vocab = ["sinal", "cabo", "ruído", "antena", "xyz", "qwe"]
prob = np.array([0.55, 0.20, 0.12, 0.08, 0.03, 0.02])

def top_k(prob, k):
    corte = np.argsort(prob)[: -1][ :k]
    nova = np.zeros_like(prob); nova[corte] = prob[corte]
    return nova / nova.sum()

def top_p(prob, p):
    ordem = np.argsort(prob)[: -1]
    acumulada = np.cumsum(prob[ordem])
    manter = ordem[:np.searchsorted(acumulada, p) + 1]
    nova = np.zeros_like(prob); nova[manter] = prob[manter]
    return nova / nova.sum()

print(f'{"token":<9}{"original":>10}{"top-k 3":>10}{"top-p 0.9":>12}')
k3, p9 = top_k(prob, 3), top_p(prob, 0.9)
for i, token in enumerate(vocab):
    print(f'{"token":<9}{"prob[i]:>10.3f"}{"k3[i]:>10.3f"}{"p9[i]:>12.3f}')
```

print("\ntop-k corta em quantidade fixa")
print("top-p corta por massa de probabilidade: adapta ao contexto")

--- SAÍDA ---

token	original	top-k 3	top-p 0.9
sinal	0.550	0.632	0.579
cabo	0.200	0.230	0.211
ruído	0.120	0.138	0.126
antena	0.080	0.000	0.084
xyz	0.030	0.000	0.000
qwe	0.020	0.000	0.000

top-k corta em quantidade fixa
top-p corta por massa de probabilidade: adapta ao contexto

--- #CÓDIGO 50.4 Perplexidade: a métrica do modelo de linguagem -----

Perplexidade é a exponencial da perda de entropia cruzada. Ela responde: entre quantos símbolos o modelo está, na prática, hesitando a cada passo. Repare que ela chega a **1.000**, ou seja, certeza absoluta a cada caractere. Isso não é excelência, é **memorização**: o corpus é pequeno e repetitivo, e o modelo o decorou. Em texto real, um bom modelo de linguagem fica entre 10 e 30, e perplexidade perto de 1 no conjunto de teste é sinal de que o teste vazou para o treino.

```
import numpy as np
import keras
from keras import layers

keras.utils.set_random_seed(0)
TEXTO = ("a antena transmite o sinal. o radio recebe o sinal. "
        "o tecnico instala a antena. o cabo conduz o sinal. "
        "a fibra conduz o sinal sem perda. ") * 120
vocab = sorted(set(TEXTO))
para_id = {c: i for i, c in enumerate(vocab)}
seq = np.array([para_id[c] for c in TEXTO])
JANELA = 24
X = np.array([seq[i:i + JANELA] for i in range(len(seq) - JANELA)])
y = seq[JANELA:]
corte = int(len(X) * 0.9)

modelo = keras.Sequential([
    layers.Input(shape=(JANELA,)),
    layers.Embedding(len(vocab), 24),
    layers.LSTM(96),
    layers.Dense(len(vocab), activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy")

def perplexidade(X, y):
    prob = modelo.predict(X, verbose=0)
    certas = prob[np.arange(len(y)), y]
    return float(np.exp(-np.log(certas + 1e-9).mean()))

print(f"perplexidade de um modelo ao acaso: {len(vocab):.1f}")
print(f"antes do treino: {perplexidade(X[corte:], y[corte:]):.3f}")

for epocas in [5, 15, 30]:
    modelo.fit(X[:corte], y[:corte], epochs=epocas -
              (0 if epocas == 5 else (5 if epocas == 15 else 15)),
              batch_size=128, verbose=0)
    print(f"após {epocas}>2} épocas: "
          f"{perplexidade(X[corte:], y[corte:]):.3f}")

print("\nperplexidade = quantas opções o modelo considera, em média")
print("quanto menor, mais certeza ele tem do próximo símbolo")

--- SAÍDA ---

perplexidade de um modelo ao acaso: 19.0
antes do treino: 19.004
após 5 épocas: 1.023
após 15 épocas: 1.002
após 30 épocas: 1.000

perplexidade = quantas opções o modelo considera, em média
quanto menor, mais certeza ele tem do próximo símbolo
```

[*] DO CARACTERE AO CHATGPT

O modelo desta seção tem menos de 50 mil parâmetros e prevê caracteres. Um LLM moderno prevê subpalavras, tem bilhões de parâmetros, usa a atenção da seção 47 em vez de LSTM e passa por uma etapa extra de alinhamento com preferências humanas. O laço de geração, porém, é exatamente este.

```
.....  
>> EXPERIMENTOS PROPOSTOS  
  
. Treine o modelo com um texto do seu domínio e gere amostras.  
. Compare a geração com temperatura 0.2, 0.8 e 1.5 e descreva o efeito.  
. Implemente a amostragem top-k dentro da função gerar.  
. Meça a perplexidade em um texto de assunto diferente do treino e explique o resultado.
```

=====

|| BLOCO XXI

SISTEMAS COM MODELOS DE LINGUAGEM

Prompting, recuperação, ajuste fino e a avaliação de sistemas que produzem texto.

=====

[51] MODELOS DE LINGUAGEM E PROMPTING

O que é um LLM, como se conversa com ele por código e quais parâmetros mudam a resposta. A seção é curta de propósito: o mecanismo já foi visto nas seções 47 e 50.

Um **modelo de linguagem grande** é o modelo autorregressivo da seção 50 levado ao extremo: bilhões de parâmetros, transformers empilhados e um corpus de trilhões de tokens. Depois do pré-treino vem o **ajuste de instrução**, que ensina o modelo a seguir pedidos, e o **alinhamento**, que o ajusta a preferências humanas.

+-- #CÓDIGO 51.1 As três fases do treino de um LLM -----

```
FASES = [
    ("PRÉ-TREINO", "trilhões de tokens da web",
     "prever o próximo token", "meses, milhares de GPUs"),
    ("AJUSTE DE INSTRUÇÃO", "milhares de pares pedido-resposta",
     "seguir instruções", "horas ou dias"),
    ("ALINHAMENTO", "comparações feitas por pessoas",
     "preferir respostas úteis e seguras", "dias"),
]

for i, (nome, dados, objetivo, custo) in enumerate(FASES, 1):
    print("+ " + "-" * 62 + "+")
    print(f"| {i}. {nome:<58} |")
    print(f"|   dados:   {dados:<47} |")
    print(f"|  objetivo: {objetivo:<47} |")
    print(f"|   custo:   {custo:<47} |")
print("+ " + "-" * 62 + "+")
print("\nquem usa a API pega o modelo depois das três fases")
print("a seção 53 mostra como adaptar sem repetir nenhuma delas")
```

--- SAÍDA ---

```
+-----+
| 1. PRÉ-TREINO |
| dados:   trilhões de tokens da web |
| objetivo: prever o próximo token   |
| custo:   meses, milhares de GPUs   |
+-----+
| 2. AJUSTE DE INSTRUÇÃO |
| dados:   milhares de pares pedido-resposta |
| objetivo: seguir instruções |
| custo:   horas ou dias |
+-----+
| 3. ALINHAMENTO |
| dados:   comparações feitas por pessoas |
| objetivo: preferir respostas úteis e seguras |
| custo:   dias |
+-----+
```

```
quem usa a API pega o modelo depois das três fases
a seção 53 mostra como adaptar sem repetir nenhuma delas
```

Chamando um modelo por código

[!] SEÇÃO NÃO EXECUTADA

Os códigos a seguir dependem de um serviço externo e de uma chave de acesso, que o ambiente de composição deste manual não possui. Eles seguem a interface usual das APIs de LLM e aparecem sem painel de saída. Todo o restante do volume foi executado.


```
+-- #CÓDIGO 51.2 A estrutura de uma conversa -----
```

Três papéis: **system** define o comportamento, **user** traz o pedido e **assistant** guarda as respostas anteriores. O modelo não tem memória: o histórico inteiro vai em toda chamada.

```
# Instale antes: pip install openai
from openai import OpenAI

cliente = OpenAI(api_key="sua-chave-aqui")

resposta = cliente.chat.completions.create(
    model="um-modelo-de-chat",
    messages=[
        {"role": "system",
         "content": "Você é um assistente técnico de "
                    "telecomunicações. Responda de forma "
                    "objetiva e cite a unidade de cada grandeza."},
        {"role": "user",
         "content": "O que significa um enlace com RSSI de "
                    "-95 dBm?"},
    ],
    temperature=0.2,      # baixa: respostas mais previsíveis
    max_tokens=300,
)

print(resposta.choices[0].message.content)
print("tokens consumidos:", resposta.usage.total_tokens)
```

```
+-- #CÓDIGO 51.3 Mantendo o histórico -----
```

A segunda pergunta só faz sentido porque a primeira está no histórico. Esse acúmulo é também o que faz o custo crescer a cada turno.

```
historico = [
    {"role": "system", "content": "Assistente de laboratório."},
]

def perguntar(cliente, texto):
    """Acrescenta a pergunta, chama o modelo e guarda a resposta."""
    historico.append({"role": "user", "content": texto})
    resposta = cliente.chat.completions.create(
        model="um-modelo-de-chat", messages=historico,
        temperature=0.3)
    conteudo = resposta.choices[0].message.content
    historico.append({"role": "assistant", "content": conteudo})
    return conteudo

# print(perguntar(cliente, "Qual a perda em espaço livre a 5 km?"))
# print(perguntar(cliente, "E se a frequência dobrar?"))

# O contexto tem limite: em conversas longas, descarte ou resuma
# as mensagens mais antigas antes de enviar.
```

Prompting

TÉCNICA	O QUE FAZER	QUANDO USA
zero-shot	só a instrução	tarefas simples e comuns
few-shot	dois a cinco exemplos no prompt	formato de saída específico
cadeia de raciocínio	pedir os passos antes da resposta	problemas com várias etapas
papel	definir quem o modelo é no <code>system</code>	manter tom e vocabulário
saída estruturada	exigir JSON com campos nomeados	quando a resposta alimenta outro programa

--- Tabela 51.1: as técnicas que mais mudam resultado.

+-- #CÓDIGO 51.4 Few-shot e saída estruturada -----

Dois exemplos ensinam o formato melhor que três parágrafos de instrução. E exigir JSON transforma a saída em algo que outro programa consegue consumir.

```
PROMPT = """Classifique o registro de manutenção em uma das
categorias: ANTENA, ENERGIA, TRANSMISSAO ou OUTRO.
Responda apenas com JSON no formato
{"categoria": "...", "confianca": 0.0}.

Registro: "cabo de alimentação rompido no poste"
Resposta: {"categoria": "ENERGIA", "confianca": 0.9}

Registro: "refletor desalinhado após vendaval"
Resposta: {"categoria": "ANTENA", "confianca": 0.95}

Registro: "%s"
Resposta: ""

registro = "perda de sincronismo no enlace de micro-ondas"
# bruto = perguntar(cliente, PROMPT % registro)
# dados = json.loads(bruto)
# print(dados["categoria"], dados["confianca"])

# Sempre valide: o modelo pode devolver texto fora do formato.
# Um json.loads dentro de try/except evita derrubar o programa.
```

+-- #CÓDIGO 51.5 Um agente: o modelo escolhendo a ferramenta -----

Este é o esqueleto de um agente, e a parte executada aqui é a que importa: o modelo **decide**, o seu código **calcula**. Nunca peça aritmética a um modelo de linguagem quando existe uma função para isso.

```
import json
import math

def perda_espaco_livre(distancia_km, frequencia_mhz):
    return (32.44 + 20 * math.log10(distancia_km)
            + 20 * math.log10(frequencia_mhz))

FERRAMENTAS = {"perda_espaco_livre": perda_espaco_livre}

# O modelo recebe a descrição das ferramentas e devolve, em JSON,
# qual chamar e com quais argumentos. O programa executa e
# devolve o resultado para o modelo redigir a resposta final.
decisao_simulada = json.dumps({
    "ferramenta": "perda_espaco_livre",
    "argumentos": {"distancia_km": 5, "frequencia_mhz": 2400},
})

pedido = json.loads(decisao_simulada)
funcao = FERRAMENTAS[pedido["ferramenta"]]
resultado = funcao(**pedido["argumentos"])

print("ferramenta escolhida:", pedido["ferramenta"])
print("argumentos:", pedido["argumentos"])
print(f"resultado: {resultado:.2f} dB")
print("\no cálculo é feito em Python, não pelo modelo")

--- SAÍDA ---

ferramenta escolhida: perda_espaco_livre
argumentos: {'distancia_km': 5, 'frequencia_mhz': 2400}
resultado: 114.02 dB

o cálculo é feito em Python, não pelo modelo
```

```
.....  
>> EXPERIMENTOS PROPOSTOS  
.  
. Escreva o prompt de sistema para um assistente da sua disciplina, definindo papel, tom e formato.  
. Monte um prompt few-shot com três exemplos para uma classificação do seu laboratório.  
. Implemente a validação da saída JSON com try/except e um valor padrão.  
. Liste três tarefas em que usar um LLM é pior que escrever a função diretamente.
```

[52] RAG: GERAÇÃO AUMENTADA POR RECUPERAÇÃO

O modelo não sabe o que está nos seus documentos, e treiná-lo de novo é caro. O RAG resolve isso buscando o trecho certo e colocando no contexto, na hora da pergunta.

+-- #CÓDIGO 52.1 O fluxo completo, em ASCII -----

```
ETAPAS = [
    ("INDEXAÇÃO", "uma vez, antes de tudo", [
        "1. quebrar os documentos em trechos",
        "2. gerar o embedding de cada trecho",
        "3. guardar os vetores em um índice",
    ]),
    ("CONSULTA", "a cada pergunta", [
        "4. gerar o embedding da pergunta",
        "5. buscar os trechos mais parecidos",
        "6. montar o prompt com os trechos recuperados",
        "7. o modelo responde usando só esse contexto",
    ]),
]

for titulo, quando, passos in ETAPAS:
    print("+ " + "-" * 58 + "+")
    print(f"| {titulo:<20}{(' ' + quando + ' '):>35} |")
    print("+ " + "-" * 58 + "+")
    for passo in passos:
        print(f"| {passo:<54} |")
    print("+ " + "-" * 58 + "+")
    print()

print("o modelo não é retreinado: o conhecimento entra pelo prompt")
```

--- SAÍDA ---

```
+-----+
| INDEXAÇÃO                (uma vez, antes de tudo) |
+-----+
|  1. quebrar os documentos em trechos                |
|  2. gerar o embedding de cada trecho                |
|  3. guardar os vetores em um índice                |
+-----+

+-----+
| CONSULTA                (a cada pergunta) |
+-----+
|  4. gerar o embedding da pergunta                |
|  5. buscar os trechos mais parecidos                |
|  6. montar o prompt com os trechos recuperados                |
|  7. o modelo responde usando só esse contexto                |
+-----+

o modelo não é retreinado: o conhecimento entra pelo prompt
```

Indexação

+-+ #CÓDIGO 52.2 Quebrando documentos em trechos -----

A sobreposição evita cortar uma informação ao meio. Trecho grande demais dilui a busca; pequeno demais perde o contexto. Entre 100 e 500 caracteres costuma funcionar.

```
DOCUMENTO = (
    "A antena Yagi possui elementos diretores e um refletor. "
    "O ganho típico fica entre 7 e 15 dBi conforme o número de "
    "elementos. A polarização acompanha a orientação dos "
    "elementos. O cabo coaxial RG-213 apresenta perda de cerca "
    "de 6 dB por 100 metros em 145 MHz. Já o RG-58 chega a 16 dB "
    "nas mesmas condições. Conectores mal crimpados são a causa "
    "mais comum de perda em instalações novas."
)
```

```
def trechos(texto, tamanho=110, sobreposicao=30):
    """Divide o texto em janelas com sobreposição."""
    palavras = texto.split()
    saida, inicio = [], 0
    while inicio < len(palavras):
        pedaco, contagem = [], 0
        i = inicio
        while i < len(palavras) and contagem < tamanho:
            pedaco.append(palavras[i])
            contagem += len(palavras[i]) + 1
            i += 1
        saida.append(" ".join(pedaco))
        if i >= len(palavras):
            break
        inicio = max(i - sobreposicao // 6, inicio + 1)
    return saida
```

```
pedacos = trechos(DOCUMENTO)
for i, pedaco in enumerate(pedacos):
    print(f"[{i}] {pedaco[:76]}...")
print(f"\n{len(pedacos)} trechos com sobreposição")
```

--- SAÍDA ---

```
[0] A antena Yagi possui elementos diretores e um refletor. O ganho típico fica ...
[1] 15 dBi conforme o número de elementos. A polarização acompanha a orientação ...
[2] elementos. O cabo coaxial RG-213 apresenta perda de cerca de 6 dB por 100 me...
[3] MHz. Já o RG-58 chega a 16 dB nas mesmas condições. Conectores mal crimpados...
[4] mais comum de perda em instalações novas....
```

5 trechos com sobreposição

+-- #CÓDIGO 52.3 Índice vetorial com TF-IDF -----

Em produção usa-se um embedding denso e um banco vetorial como FAISS ou Chroma. O TF-IDF cumpre o mesmo papel aqui, roda sem baixar nada e deixa a mecânica visível.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

BASE = [
    "A antena Yagi tem ganho entre 7 e 15 dBi conforme o número "
    "de elementos diretores.",
    "O cabo RG-213 apresenta perda de 6 dB por 100 metros em "
    "145 MHz.",
    "O cabo RG-58 apresenta perda de 16 dB por 100 metros em "
    "145 MHz.",
    "Conectores mal crimpados são a causa mais comum de perda em "
    "instalações novas.",
    "A polarização da Yagi acompanha a orientação dos elementos.",
    "O rádio deve ser desligado antes de qualquer intervenção na "
    "torre.",
]

vetorizador = TfidfVectorizer()
indice = vetorizador.fit_transform(BASE)

print("trechos indexados:", indice.shape[0])
print("dimensão do vetor:", indice.shape[1])
print("termos:", vetorizador.get_feature_names_out()[:8], "...")
```

--- SAÍDA ---

```
trechos indexados: 6
dimensão do vetor: 49
termos: ['100' '145' '15' '16' '213' '58' 'acompanha' 'antena'] ...
```

+-----
Consulta

+-- #CÓDIGO 52.4 Recuperando os trechos relevantes -----

Repare na terceira pergunta: a similaridade despenca. Esse número é o seu detector de pergunta fora do escopo, e usá-lo é o que impede o sistema de inventar resposta.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

BASE = [
    "A antena Yagi tem ganho entre 7 e 15 dBi conforme o número "
    "de elementos diretores.",
    "O cabo RG-213 apresenta perda de 6 dB por 100 metros em 145 MHz.",
    "O cabo RG-58 apresenta perda de 16 dB por 100 metros em 145 MHz.",
    "Conectores mal crimpados são a causa mais comum de perda em "
    "instalações novas.",
    "A polarização da Yagi acompanha a orientação dos elementos.",
    "O rádio deve ser desligado antes de qualquer intervenção na torre.",
]

vetorizador = TfidfVectorizer()
indice = vetorizador.fit_transform(BASE)

def recuperar(pergunta, k=2):
    vetor = vetorizador.transform([pergunta])
    similar = cosine_similarity(vetor, indice)[0]
    ordem = np.argsort(similar)[::-1][:k]
    return [(BASE[i], float(similar[i])) for i in ordem]

for pergunta in ["qual a perda do cabo RG-58?",
                 "quanto de ganho tem uma Yagi?",
                 "qual a receita de bolo de cenoura?"]:
    print(f"P: {pergunta}")
    for trecho, nota in recuperar(pergunta):
        print(f"    [{nota:.3f}] {trecho[:62]}...")
    print()
```

--- SAÍDA ---

```
P: qual a perda do cabo RG-58?
[0.552] O cabo RG-58 apresenta perda de 16 dB por 100 metros em 145 MH...
[0.377] O cabo RG-213 apresenta perda de 6 dB por 100 metros em 145 MH...

P: quanto de ganho tem uma Yagi?
[0.526] A antena Yagi tem ganho entre 7 e 15 dBi conforme o número de ...
[0.156] A polarização da Yagi acompanha a orientação dos elementos....

P: qual a receita de bolo de cenoura?
[0.178] O cabo RG-213 apresenta perda de 6 dB por 100 metros em 145 MH...
[0.168] O rádio deve ser desligado antes de qualquer intervenção na to...
```

+-----

+-+ #CÓDIGO 52.5 Montando o prompt com o contexto -----

Três instruções fazem quase todo o trabalho: responder só pelo contexto, admitir quando não souber e citar a fonte. Sem elas, o modelo completa as lacunas com invenção.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

BASE = [
    "A antena Yagi tem ganho entre 7 e 15 dBi conforme o número "
    "de elementos diretores.",
    "O cabo RG-213 apresenta perda de 6 dB por 100 metros em 145 MHz.",
    "O cabo RG-58 apresenta perda de 16 dB por 100 metros em 145 MHz.",
    "Conectores mal crimpados são a causa mais comum de perda em "
    "instalações novas.",
]

vetorizador = TfidfVectorizer()
indice = vetorizador.fit_transform(BASE)
LIMIAR = 0.15

def montar_prompt(pergunta, k=2):
    vetor = vetorizador.transform([pergunta])
    similar = cosine_similarity(vetor, indice)[0]
    ordem = [i for i in np.argsort(similar)[::-1][:k]
              if similar[i] >= LIMIAR]

    if not ordem:
        return None, "sem contexto suficiente na base"

    contexto = "\n".join(f"[{n + 1}] {BASE[i]}"
                        for n, i in enumerate(ordem))

    prompt = (
        "Responda apenas com base no contexto abaixo. "
        "Se a resposta não estiver nele, diga que não sabe. "
        "Cite o número do trecho usado.\n\n"
        f"CONTEXTO:\n{contexto}\n\n"
        f"PERGUNTA: {pergunta}\nRESPOSTA:"
    )
    return prompt, None

prompt, aviso = montar_prompt("qual a perda do RG-213?")
print(prompt)
print("\n" + "=" * 60 + "\n")
prompt, aviso = montar_prompt("como faço um bolo?")
print("pergunta fora do escopo ->", aviso)
```

--- SAÍDA ---

Responda apenas com base no contexto abaixo. Se a resposta não estiver nele, diga que não sabe. Cite o número do trecho usado.

CONTEXTO:

[1] O cabo RG-213 apresenta perda de 6 dB por 100 metros em 145 MHz.
[2] O cabo RG-58 apresenta perda de 16 dB por 100 metros em 145 MHz.

PERGUNTA: qual a perda do RG-213?

RESPOSTA:

=====

pergunta fora do escopo -> sem contexto suficiente na base

+-- #CÓDIGO 52.6 Avaliando a recuperação -----

Antes de culpar o modelo por uma resposta ruim, meça a recuperação: se o trecho certo não entrou no contexto, nenhum LLM do mundo acerta a resposta. Mas não se engane com o resultado acima. Com cinco trechos e perguntas que repetem as palavras do texto (RG-213, Yagi), acertar 100% em primeiro lugar é o esperado, e o número não informa nada. Em uma base real, com milhares de trechos e perguntas que **parafraseiam** em vez de repetir, o TF-IDF começa a falhar: ele casa palavras, não sentido. É esse limite que justifica trocá-lo por embeddings treinados.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

BASE = [
    "A antena Yagi tem ganho entre 7 e 15 dBi conforme o número "
    "de elementos diretores.",
    "O cabo RG-213 apresenta perda de 6 dB por 100 metros em 145 MHz.",
    "O cabo RG-58 apresenta perda de 16 dB por 100 metros em 145 MHz.",
    "Conectores mal crimpados são a causa mais comum de perda em "
    "instalações novas.",
    "A polarização da Yagi acompanha a orientação dos elementos.",
]

# gabarito: qual trecho responde cada pergunta
TESTES = [
    ("qual a perda do cabo RG-213", 1),
    ("perda do RG-58 em 145 MHz", 2),
    ("ganho da antena Yagi", 0),
    ("causa comum de perda em instalação", 3),
    ("como está polarizada a Yagi", 4),
]

vetorizador = TfidfVectorizer()
indice = vetorizador.fit_transform(BASE)

def posicoes(pergunta):
    similar = cosine_similarity(
        vetorizador.transform([pergunta]), indice)[0]
    return list(np.argsort(similar)[::-1])

for k in [1, 2, 3]:
    acertos = sum(1 for pergunta, certo in TESTES
                  if certo in posicoes(pergunta)[:k])
    print(f"acerto@{k}: {acertos}/{len(TESTES)} "
          f"f"= {acertos / len(TESTES):.0%}")

# posicao media do trecho correto (1 e o ideal)
rr = [1 / (posicoes(p).index(c) + 1) for p, c in TESTES]
print(f"\nMRR (posição recíproca média): {np.mean(rr):.4f}")

--- SAÍDA ---

acerto@1: 5/5 = 100%
acerto@2: 5/5 = 100%
acerto@3: 5/5 = 100%

MRR (posição recíproca média): 1.0000
```

[*] RAG OU AJUSTE FINO?

Use **RAG** quando o conhecimento muda, precisa ser citado ou é grande demais para caber no treino: manuais, normas, documentação interna. Use **ajuste fino** quando o que muda é o comportamento, o formato ou o vocabulário. Os dois se combinam, e na dúvida comece pelo RAG, que é mais barato e mais fácil de auditar.

>> EXPERIMENTOS PROPOSTOS

- Monte um RAG sobre as normas ou apostilas da sua disciplina.
- Varie o tamanho do trecho e meça o efeito no acerto@1.
- Implemente o limiar de similaridade e teste com perguntas fora do escopo.
- Substitua o TF-IDF pelos embeddings treinados na seção 46 e compare o MRR.

[53] AJUSTE FINO, LORA E QUANTIZAÇÃO

Três formas de adaptar um modelo grande sem repetir o pré-treino: treinar tudo, treinar quase nada, ou apenas encolher o que já existe.

ESTRATÉGIA	PARÂMETROS TREINADOS	QUANDO USAR
prompting	nenhum	primeira tentativa, sempre
RAG	nenhum	conhecimento próprio e mutável
ajuste de cabeça	última camada	tarefa nova, poucos dados
LoRA	menos de 1%	comportamento novo, dados médios
ajuste completo	todos	muitos dados e muito orçamento

+-- Tabela 53.1: da mais barata à mais cara.

+-- #CÓDIGO 53.1 Congelar e treinar só a cabeça

O corpo congelado vira um extrator de características. Com 120 amostras e 49 parâmetros treináveis já se resolve uma tarefa nova.

```
import numpy as np
import keras
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")

# modelo "grande", ja treinado em uma tarefa anterior
base = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(96, activation="relu"),
    layers.Dense(48, activation="relu"),
])
antigo = keras.Sequential([base, layers.Dense(10, activation="softmax")])
antigo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
antigo.fit(X, y, epochs=20, batch_size=64, verbose=0)

# tarefa nova: apenas dizer se o dígito é par
y_novo = (y % 2 == 0).astype("float32")
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y_novo, train_size=120, random_state=0, stratify=y_novo)

base.trainable = False
adaptado = keras.Sequential([base, layers.Dense(1, activation="sigmoid")])
adaptado.compile("adam", "binary_crossentropy", metrics=["accuracy"])
adaptado.fit(X_tr, y_tr, epochs=40, batch_size=16, verbose=0)

treinaveis = sum(np.prod(v.shape) for v in adaptado.trainable_weights)
total = adaptado.count_params()
print(f"parâmetros totais:      {total:>8}")
print(f"parâmetros treinados:  {treinaveis:>8} "
      f"({100 * treinaveis / total:.2f}%)")
print(f"acurácia na tarefa nova: "
      f"{adaptado.evaluate(X_te, y_te, verbose=0)[1]:.4f}")
```

--- SAÍDA ---

```
parâmetros totais:      10945
parâmetros treinados:    49 (0.45%)
acurácia na tarefa nova: 0.8605
```

LoRA

Ajustar uma camada densa significa alterar uma matriz **W** grande. A ideia do **LoRA** é não tocar em **W**: congela-se a original e aprende-se apenas uma correção de **posto baixo**, o produto de duas matrizes finas. A saída vira **W x + B A**

x , e só A e B são treinados.

+-- #CÓDIGO 53.2 Implementando LoRA do zero -----

Iniciar B com zeros garante que o adaptador nasce neutro, sem estragar o que o modelo já sabia. É por isso que o LoRA é seguro mesmo com poucos dados.

```
import numpy as np
import keras
from keras import layers, ops

class LoRA(layers.Layer):
    """Adaptador de posto baixo sobre uma camada densa congelada."""

    def __init__(self, densa, posto=2, escala=1.0, **kwargs):
        super().__init__(**kwargs)
        self.densa = densa
        self.densa.trainable = False      # a original nao muda
        self.posto = posto
        self.escala = escala

    def build(self, forma):
        entrada = int(forma[-1])
        saida = int(self.densa.units)
        self.A = self.add_weight(
            shape=(entrada, self.posto), initializer="glorot_uniform",
            trainable=True, name="A")
        self.B = self.add_weight(
            shape=(self.posto, saida), initializer="zeros",
            trainable=True, name="B")

    def call(self, x):
        return self.densa(x) + self.escala * ops.matmul(
            ops.matmul(x, self.A), self.B)

keras.utils.set_random_seed(0)
densa = layers.Dense(96)
densa.build((None, 64))
adaptador = LoRA(densa, posto=2)
adaptador.build((None, 64))

originais = int(np.prod(densa.kernel.shape) + densa.bias.shape[0])
novos = int(np.prod(adaptador.A.shape) + np.prod(adaptador.B.shape))

print(f"parâmetros da camada original: {originais:>7}")
print(f"parâmetros do adaptador LoRA: {novos:>7}")
print(f"proporção treinada: {100 * novos / originais:.2f}%")
print("\nB começa em zero: no instante inicial o modelo é idêntico")
```

--- SAÍDA ---

```
parâmetros da camada original:    6240
parâmetros do adaptador LoRA:      320
proporção treinada: 5.13%
```

B começa em zero: no instante inicial o modelo é idêntico

+-+ #CÓDIGO 53.3 LoRA aprendendo uma tarefa nova -----

O ponto central está na última linha: o modelo original continua funcionando. Um adaptador por tarefa, alguns kilobytes cada, contra uma cópia inteira do modelo por tarefa.

```
import numpy as np
import keras
from keras import layers, ops
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

class LoRA(layers.Layer):
    def __init__(self, densa, posto=4, **kwargs):
        super().__init__(**kwargs)
        self.densa = densa; self.densa.trainable = False
        self.posto = posto
    def build(self, forma):
        self.A = self.add_weight(shape=(int(forma[-1]), self.posto),
                                initializer="glorot_uniform", name="A")
        self.B = self.add_weight(shape=(self.posto, int(self.densa.units)),
                                initializer="zeros", name="B")
    def call(self, x):
        return self.densa(x) + ops.matmul(ops.matmul(x, self.A), self.B)

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")

corpo = layers.Dense(96, activation=None)
base = keras.Sequential([layers.Input(shape=(64,)), corpo,
                        layers.Activation("relu"),
                        layers.Dense(10, activation="softmax")])
base.compile("adam", "sparse_categorical_crossentropy",
            metrics=["accuracy"])
base.fit(X, y, epochs=20, batch_size=64, verbose=0)
print(f"tarefa original (10 dígitos): "
      f"{base.evaluate(X, y, verbose=0)[1]:.4f}")

y_novo = (y % 2 == 0).astype("float32")
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y_novo, train_size=200, random_state=0, stratify=y_novo)

adaptado = keras.Sequential([
    layers.Input(shape=(64,)),
    LoRA(corpo, posto=4),
    layers.Activation("relu"),
    layers.Dense(1, activation="sigmoid"),
])
adaptado.compile("adam", "binary_crossentropy", metrics=["accuracy"])
adaptado.fit(X_tr, y_tr, epochs=40, batch_size=16, verbose=0)

treinaveis = sum(int(np.prod(v.shape))
                 for v in adaptado.trainable_weights)
print(f"\nparâmetros treinados: {treinaveis}")
print(f"acurácia na tarefa nova: "
      f"{adaptado.evaluate(X_te, y_te, verbose=0)[1]:.4f}")
print(f"tarefa original ainda intacta: "
      f"{base.evaluate(X, y, verbose=0)[1]:.4f}")
```

--- SAÍDA ---

tarefa original (10 dígitos): 0.9777

parâmetros treinados: 737

acurácia na tarefa nova: 0.9073

tarefa original ainda intacta: 0.9777

Quantização

+-+ #CÓDIGO 53.4 O custo de guardar os pesos -----

Quantizar troca a precisão numérica dos pesos por memória. De 32 para 4 bits, o modelo encolhe oito vezes, com perda de qualidade que costuma ser pequena.

```
import numpy as np

def memoria(parametros, bits):
    return parametros * bits / 8 / 1e9      # em GB

MODELOS = [("pequeno", 1e9), ("médio", 7e9), ("grande", 70e9)]

print(f'{"modelo":<10}{ "parâmetros":>13}{ "fp32":>9}{ "fp16":>9}'
      f'{"int8":>9}{ "int4":>9}')
for nome, n in MODELOS:
    linha = f'{"nome":<10}{n / 1e9:>11.0f} B'
    for bits in [32, 16, 8, 4]:
        linha += f'{"memoria(n, bits):>9.1f}"
    print(linha + "      GB")

print("\numa GPU comum tem 8 a 24 GB")
print("é a quantização que faz um modelo de 7 B caber nela")
```

--- SAÍDA ---

modelo	parâmetros	fp32	fp16	int8	int4	
pequeno	1 B	4.0	2.0	1.0	0.5	GB
médio	7 B	28.0	14.0	7.0	3.5	GB
grande	70 B	280.0	140.0	70.0	35.0	GB

uma GPU comum tem 8 a 24 GB

é a quantização que faz um modelo de 7 B caber nela

+-- #CÓDIGO 53.5 Medindo o efeito da quantização -----

Até 8 bits a perda costuma ser desprezível; em 4 ela aparece; em 2 o modelo desmonta. Meça sempre no seu caso antes de escolher o nível.

```
import numpy as np
import keras
from keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(0)
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

modelo = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(96, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
modelo.compile("adam", "sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_tr, y_tr, epochs=25, batch_size=32, verbose=0)
print(f"original (float32): {modelo.evaluate(X_te, y_te, verbose=0)[1]:.4f}")

originais = [v.numpy().copy() for v in modelo.weights]

def quantizar(valores, bits):
    """Simula a redução de precisão dos pesos."""
    niveis = 2 ** bits - 1
    minimo, maximo = valores.min(), valores.max()
    passo = (maximo - minimo) / niveis
    return np.round((valores - minimo) / passo) * passo + minimo

for bits in [8, 4, 2]:
    for v, original in zip(modelo.weights, originais):
        v.assign(quantizar(original, bits))
    acuracia = modelo.evaluate(X_te, y_te, verbose=0)[1]
    print(f"quantizado em {bits} bits: {acuracia:.4f}")

for v, original in zip(modelo.weights, originais):
    v.assign(original)

--- SAÍDA ---
original (float32): 0.9689
quantizado em 8 bits: 0.9689
quantizado em 4 bits: 0.9689
quantizado em 2 bits: 0.8289
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Adapte o modelo da seção 40 para uma classe nova de sinal usando apenas a cabeça.
- . Varie o posto do LoRA entre 1 e 16 e compare acurácia contra número de parâmetros.
- . Verifique se o modelo original continua intacto após treinar o adaptador.
- . Descubra em quantos bits o seu modelo começa a perder mais de 1% de acurácia.

=====

|| BLOCO XXII

APÊNDICES

Consulta rápida de IA generativa.

=====

[G] CARTÃO DE CONSULTA E GLOSSÁRIO

A Parte IV inteira em duas páginas, e o vocabulário em inglês que você vai encontrar na documentação.

+-- #CÓDIGO G.1 Resumo da Parte IV -----

```
# ---- tokenizacao (secao 45) -----
import tiktoken
cod = tiktoken.get_encoding("cl100k_base")
fichas = cod.encode("texto"); cod.decode(fichas)
# alternativas: sentencepiece, tokenizers (HuggingFace)

# ---- embeddings (secao 46) -----
from keras import layers
layers.Embedding(tamanho_vocab, dimensao)      # treinado junto
# busca semantica sem rede neural:
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics.pairwise import cosine_similarity

# ---- atencao (secao 47) -----
# Atencao(Q,K,V) = softmax(Q K^T / raiz(d)) V
layers.MultiHeadAttention(num_heads=8, key_dim=64)(x, x)
layers.MultiHeadAttention(...)(x, x, use_causal_mask=True) # geracao
layers.LayerNormalization(epsilon=1e-6)(x + atencao)      # residual

# ---- familias generativas (secoes 48 a 50) -----
# VAE      : codifica para uma distribuicao, amostra, decodifica
# GAN      : gerador contra discriminador
# difusao  : aprende a remover ruido, passo a passo
# autorregressivo: prediz o proximo simbolo, realimenta a saida

# ---- geracao (secao 50) -----
prob = modelo.predict(contexto)[0]
prob = prob ** (1 / temperatura)          # < 1 conservador, > 1 criativo
prob = prob / prob.sum()
proximo = rng.choice(len(prob), p=prob)    # amostragem
# top-k: zera tudo fora dos k maiores; top-p: mantem massa acumulada p

# ---- RAG (secao 52) -----
# 1. quebrar em trechos  2. indexar  3. recuperar
# 4. montar o prompt com o contexto  5. gerar  6. citar a fonte
import faiss
indice = faiss.IndexFlatIP(dimensao); indice.add(vetores)
distancia, posicao = indice.search(consulta, k=4)

# ---- ajuste fino (secao 53) -----
corpo.trainable = False                  # congela o que ja sabe
# LoRA: W + (A @ B), com A e B de posto baixo e so elas treinaveis
# quantizacao: float32 -> int8 reduz o arquivo em cerca de 75%
```

Glossário

INGLÊS	PORTUGUÊS	O QUE É
token	ficha, token	a menor unidade que o modelo lê
embedding	vetor de representação	vetor aprendido em que a distância significa semelhança
attention	atenção	média ponderada sobre todas as posições
self-attention	autoatenção	a sequência atendendo a si mesma
causal mask	máscara causal	impede olhar para o futuro durante a geração
context window	janela de contexto	quantos tokens o modelo enxerga de uma vez
temperature	temperatura	quanto a amostragem se afasta do mais provável
top-k / top-p	top-k / núcleo	cortes que limitam de onde se sorteia
perplexity	perplexidade	entre quantos símbolos o modelo hesita
hallucination	alucinação	resposta fluente e falsa, dita com confiança
grounding	ancoragem	prender a resposta a uma fonte verificável
retrieval	recuperação	buscar os trechos que respondem à pergunta
chunking	fragmentação	quebrar o documento em pedaços indexáveis
prompt	instrução, prompt	o texto de entrada que condiciona a resposta
few-shot	com poucos exemplos	ensinar o formato dando exemplos no próprio prompt
fine-tuning	ajuste fino	continuar o treino em dados da sua tarefa
LoRA	adaptação de posto baixo	treinar duas matrizes pequenas em vez do modelo todo
quantization	quantização	guardar os pesos com menos bits
latent space	espaço latente	a representação comprimida onde o modelo trabalha
diffusion	difusão	gerar removendo ruído em muitos passos

+- Tabela G.1: vocabulário da IA generativa.

Qual família para qual tarefa

VOCÊ QUER	FAMÍLIA INDICADA	SEÇÃO
gerar texto	autorregressivo (transformer decodificador)	47, 50
gerar imagem de qualidade	difusão	49
gerar rápido, aceitando menos qualidade	GAN	48
comprimir e amostrar de um espaço contínuo	VAE	48
responder com base em documentos seus	RAG	52
adaptar um modelo grande com pouco dado	LoRA e ajuste fino	53
buscar por significado, não por palavra	embeddings e índice vetorial	46, 52
rodar em máquina modesta	quantização	53

+- Tabela G.2: escolhendo o caminho.

Antes de publicar algo generativo

1. A saída pode ser verificada por alguém? Fluência não é evidência de correção.
2. Existe uma fonte citada, ou o sistema está inventando com confiança?
3. O que acontece quando a resposta certa não está na base?
4. Os dados de treino ou de recuperação podem ser publicados, do ponto de vista de licença e de privacidade?
5. Quem responde por um erro: o autor do modelo, quem publicou o sistema ou quem leu a resposta?
5. Existe um caminho para a pessoa contestar a saída e falar com um humano?

[!] O QUE ESTES MODELOS NÃO FAZEM

Um modelo de linguagem estima qual símbolo vem depois. Ele não consulta a verdade, não sabe o que ignora e não distingue o que aprendeu do que está inventando. As respostas erradas chegam com a mesma fluência das certas, e é justamente isso que as torna perigosas em uso profissional. RAG melhora a situação porque prende a resposta a um documento, mas não a resolve: se a recuperação trouxer o trecho errado, a geração erra com a mesma segurança.

Para continuar

- * Documentação do Keras em keras.io, seção *KerasHub*, com transformers prontos para uso.
- * HuggingFace: modelos, conjuntos de dados e a biblioteca [transformers](https://huggingface.co/transformers).
- * O artigo *Attention Is All You Need* (2017), que introduziu a arquitetura da seção 47.
- * Para rodar modelos grandes sem instalar nada: Google Colab, com GPU por sessão.
- * Temas naturais depois deste volume: modelos multimodais, agentes com ferramentas, destilação e avaliação automática de sistemas generativos.

[*] OS QUATRO VOLUMES

A Parte I ensina a linguagem, a II o ferramental numérico, a III o processo de desenvolvimento de um modelo e a IV a geração. Cada volume assume o anterior, mas os apêndices A a G foram escritos para serem consultados soltos, durante uma prova ou um laboratório.

Manual de Experimentos · Linguagem Python · Parte IV · Ramon Mayor Martins · IFSC Câmpus São José. Continuação das Partes I (linguagem, seções 1 a 16), II (pacotes científicos, 17 a 24) e III (aprendizado de máquina, 25 a 44). Composto em Courier Prime. Exemplos executados em Python 3.12 com TensorFlow 2.21, Keras 3.15 e scikit-learn 1.8.